AI VIETNAM
@aivietnam.edu.vn

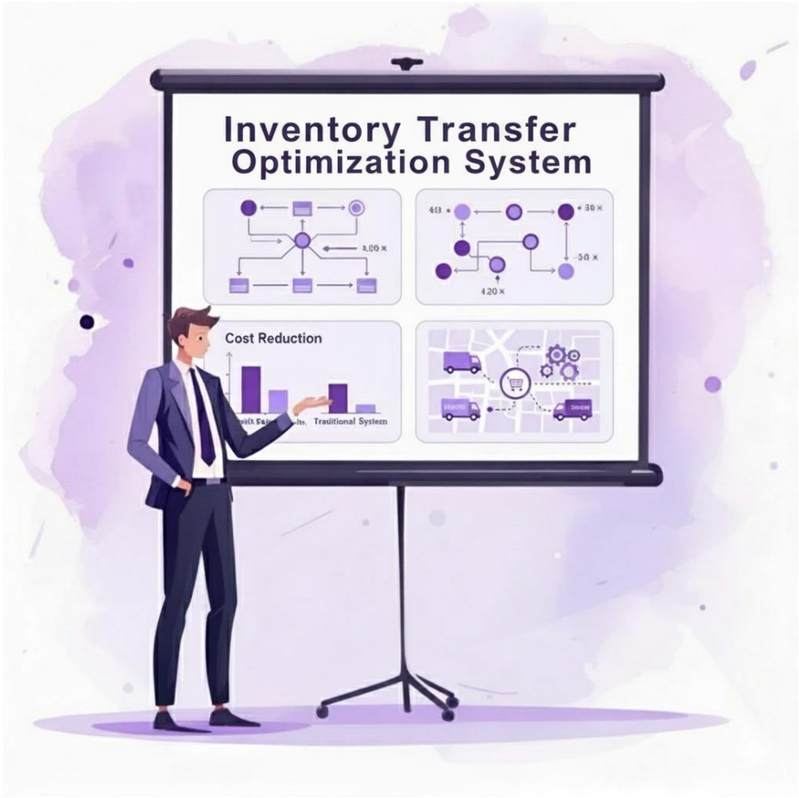
BÀI GIẢNG: XÂY DỰNG HỆ THỐNG TỐI ƯU HÓA PHÂN BỔ HÀNG HÓA

Inventory Transfer Optimization System

Lecturer: Nguyễn Thái Hà

TA: Nguyễn Thọ Anh Khoa

STA: Nguyễn Phúc Thịnh



Source Code Web Quiz

Outline bài giảng

PHẦN 1: Giới thiệu & Phân tích bài toán

- Tổng quan về tối ưu hóa
- Phân tích vấn đề hiện tại
- Xác định mục tiêu và phạm vi

PHẦN 2: Thiết kế & Triển khai mô hình tối ưu

- Lựa chọn thuật toán và công cụ
- Xây dựng cấu trúc mô hình
- Triển khai và kiểm thử mô hình

PHẦN 3: Triển khai, Đánh giá & Ứng dụng nâng cao

- Tích hợp và vận hành hệ thống
- Đánh giá hiệu suất và tối ưu
- Mở rộng và ứng dụng trong tương lai

Outline bài giảng

PHẦN 1: Giới thiệu & Phân tích bài toán

- Tổng quan về tối ưu hóa
- Phân tích vấn đề hiện tại
- Xác định mục tiêu và phạm vi

PHẦN 2: Thiết kế & Triển khai mô hình tối ưu

- Lựa chọn thuật toán và công cụ
- Xây dựng cấu trúc mô hình
- Triển khai và kiểm thử mô hình

PHẦN 3: Triển khai, Đánh giá & Ứng dụng nâng cao

- Tích hợp và vận hành hệ thống
- Đánh giá hiệu suất và tối ưu
- Mở rộng và ứng dụng trong tương lai

Giới thiệu dự án

Bài toán

Phân bổ hàng hóa giữa các cửa hàng trong chuỗi bán lẻ

Mục tiêu

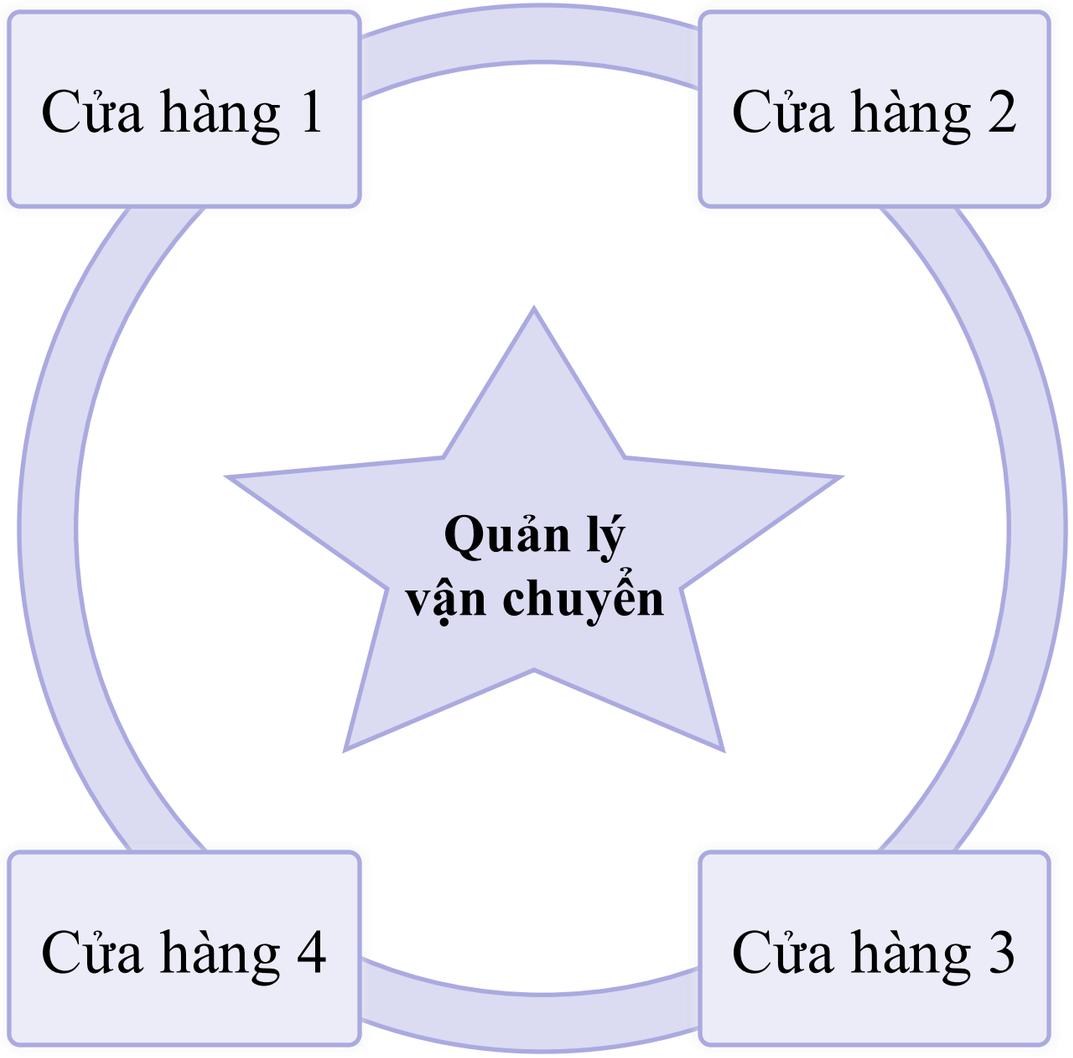
Giảm tồn kho
Tránh thiếu hàng
Tối ưu chi phí vận chuyển

Phương pháp

Ứng dụng kỹ thuật **Optimization** và **Data Science** để hỗ trợ quyết định

Thực tiễn

Mô phỏng dự án đã triển khai trên chuỗi cửa hàng thời trang lớn của Nhật. **600+** cửa hàng tại Nhật, **600+** cửa hàng tại Trung Quốc, và rất nhiều cửa hàng trên Thế Giới



Tại sao cần tối ưu hóa phân bổ hàng hóa?

AI VIETNAM
@aivietnam.edu.vn

Vấn đề thực tế

Một số cửa hàng thừa hàng, gây lãng phí; số khác lại thiếu hàng, bỏ lỡ doanh thu.

Cửa hàng A tồn đọng 100 sản phẩm, cửa hàng B hết sạch

Chi phí cao

Tồn kho quá nhiều dẫn đến chi phí lưu trữ lớn. Vận chuyển không hiệu quả làm tăng chi phí hoạt động.

Tồn kho dư thừa gây lãng phí vốn lưu động

Quyết định chậm

Việc phân bổ hàng hóa phụ thuộc nhiều vào kinh nghiệm cá nhân, dẫn đến quyết định chậm và thiếu chính xác.

Quyết định thủ công dễ bị ảnh hưởng bởi yếu tố chủ quan

Ứng dụng Data Science

Data-driven có thể tự động hóa, phân tích dữ liệu và tối ưu hóa quy trình phân bổ, mang lại hiệu quả vượt trội.

Hệ thống tự động đề xuất chuyển hàng tối ưu

Bối cảnh thực tế: Chuỗi cửa hàng bán lẻ

Vị trí & Nhu cầu đa dạng

Mỗi cửa hàng có vị trí địa lý và đặc điểm khách hàng riêng biệt.

Ví dụ: Cửa hàng ở khu văn phòng có nhu cầu khác với cửa hàng ở khu dân cư đông đúc.

Mất cân bằng tồn kho

Một số cửa hàng thừa hàng, gây lãng phí; số khác lại thiếu hàng, bỏ lỡ doanh thu.

Ví dụ: Cửa hàng A tồn đọng 100 sản phẩm, trong khi Cửa hàng B hết sạch mặt hàng đó và khách hàng phải bỏ đi

Chi phí vận chuyển cao

Khoảng cách và phương thức vận chuyển khác nhau dẫn đến chi phí không đồng đều.

Ví dụ: Chuyển hàng từ kho tổng đến cửa hàng xa 100km tốn kém hơn nhiều so với cửa hàng chỉ cách 5km.

Nhu cầu giải pháp thông minh

Cần một hệ thống tự động phân tích dữ liệu, dự báo nhu cầu và đưa ra quyết định phân bổ tối ưu.

Ví dụ: Hệ thống tự động đề xuất chuyển 30 sản phẩm từ Cửa hàng A sang Cửa hàng B để tối ưu doanh thu và giảm chi phí lưu kho.

4 thách thức lớn khi quản lý chuỗi cửa hàng

1. **Dữ liệu thay đổi liên tục:** Nhu cầu khách hàng không ổn định theo mùa, sự kiện và xu hướng thị trường, gây khó khăn trong việc dự báo và lập kế hoạch.
   - *Ví dụ: Doanh số kem tăng vọt vào mùa hè nhưng giảm mạnh vào mùa đông, trong khi đồ dùng học tập lại bán chạy trước khai giảng.*
2. **Quyết định thủ công, dễ sai sót:** Việc dựa vào kinh nghiệm cá nhân của người quản lý có thể dẫn đến các quyết định chủ quan, chậm trễ, thiếu chính xác.
   - *Ví dụ: Người quản lý có thể đặt hàng quá nhiều do lo sợ thiếu hàng, hoặc bỏ lỡ cơ hội bán hàng do đánh giá sai nhu cầu.*
3. **Khó cân bằng chi phí-hiệu quả:** Thách thức trong việc giảm chi phí tồn kho và vận chuyển mà vẫn đảm bảo đáp ứng đầy đủ nhu cầu của khách hàng, tránh tình trạng hết hàng.
   - *Ví dụ: Giảm chi phí vận chuyển bằng cách gom đơn hàng lớn có thể làm chậm thời gian giao hàng, ảnh hưởng đến sự hài lòng của khách.*
4. **Đa yếu tố phức tạp:** Phải xem xét và tối ưu đồng thời nhiều yếu tố như tồn kho, chi phí vận chuyển, dự báo nhu cầu theo từng cửa hàng, và khoảng cách địa lý.
   - *Ví dụ: Quyết định chuyển hàng từ cửa hàng A sang B cần tính toán cả lượng tồn kho hiện tại, chi phí vận chuyển, và dự báo nhu cầu của cả hai cửa hàng trong thời gian tới.*

Impact của dự án

AI VIETNAM
@aivietnam.edu.vn

Hệ thống tối ưu hóa mang lại lợi ích thiết thực cho doanh nghiệp

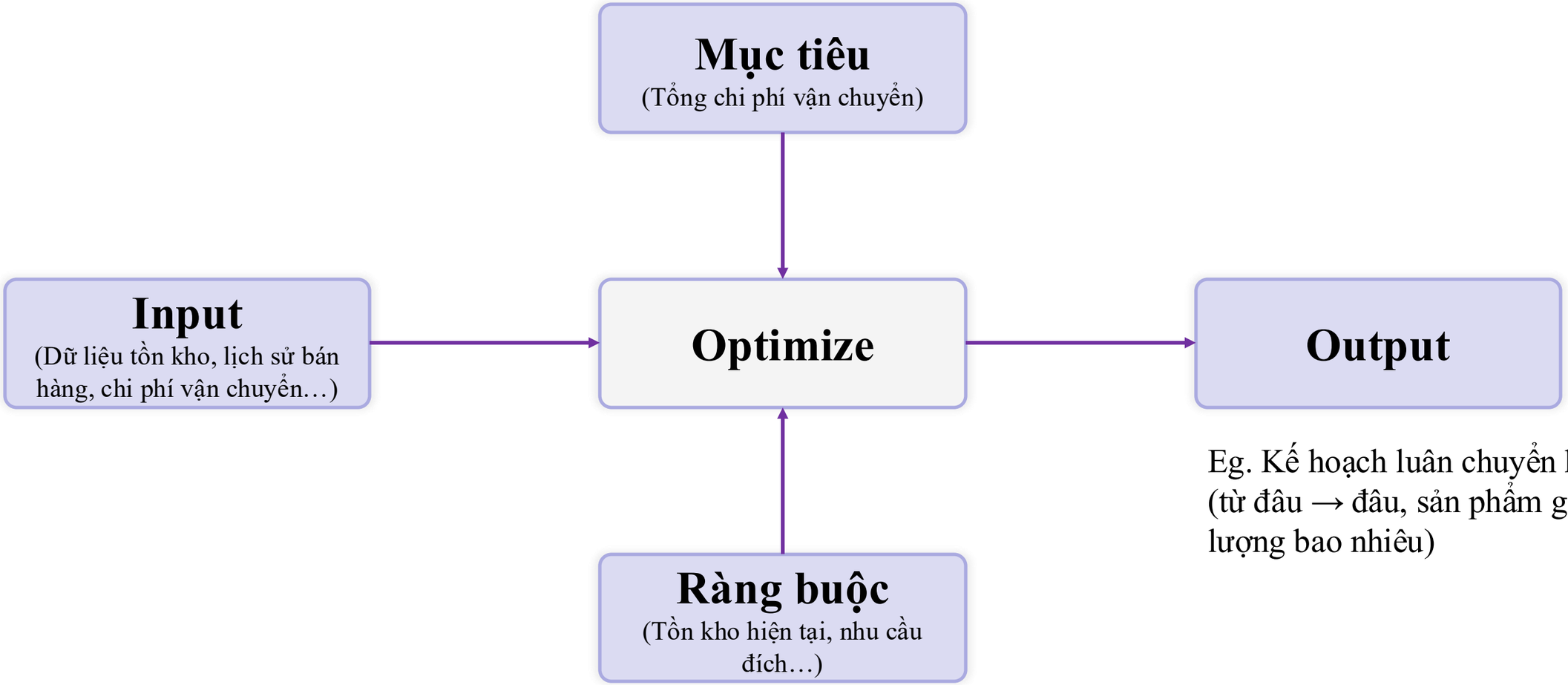
xxx%	xxx%	xxx%	xxx%
Giảm chi phí	**Tự động hóa**	**Giảm thời gian**	**Độ chính xác**
• Tiết kiệm chi phí tồn kho và vận chuyển	• Giảm can thiệp thủ công, tăng tốc độ xử lý	• Xử lý nhanh hơn so với phương pháp truyền thống	• Dự báo và quyết định chính xác hơn

Tiêu chí lựa chọn phương pháp

- Tăng khả năng đáp ứng nhu cầu
- Giảm rủi ro thiếu hàng, mất doanh thu
- Nâng cao hiệu quả chuỗi cung ứng
- Phát triển kỹ năng AI/Optimization

Định nghĩa bài toán tối ưu

Bài toán tối ưu là bài toán tìm ra phương án tốt nhất theo một tiêu chí nhất định trong điều kiện các ràng buộc cho trước.



Eg. Kế hoạch luân chuyển hàng (từ đâu → đâu, sản phẩm gì, số lượng bao nhiêu)

Flow triển khai hệ thống tối ưu hóa

1. Thu thập dữ liệu

Tập hợp các dữ liệu cần thiết như tồn kho hiện tại, lịch sử bán hàng, thông tin khoảng cách và chi phí vận chuyển.

2. Phân tích dữ liệu

Tính toán Days of Supply, xác định lượng tồn kho thừa (excess inventory) và thiếu hụt (needed inventory) tại các điểm.

3. Chạy thuật toán

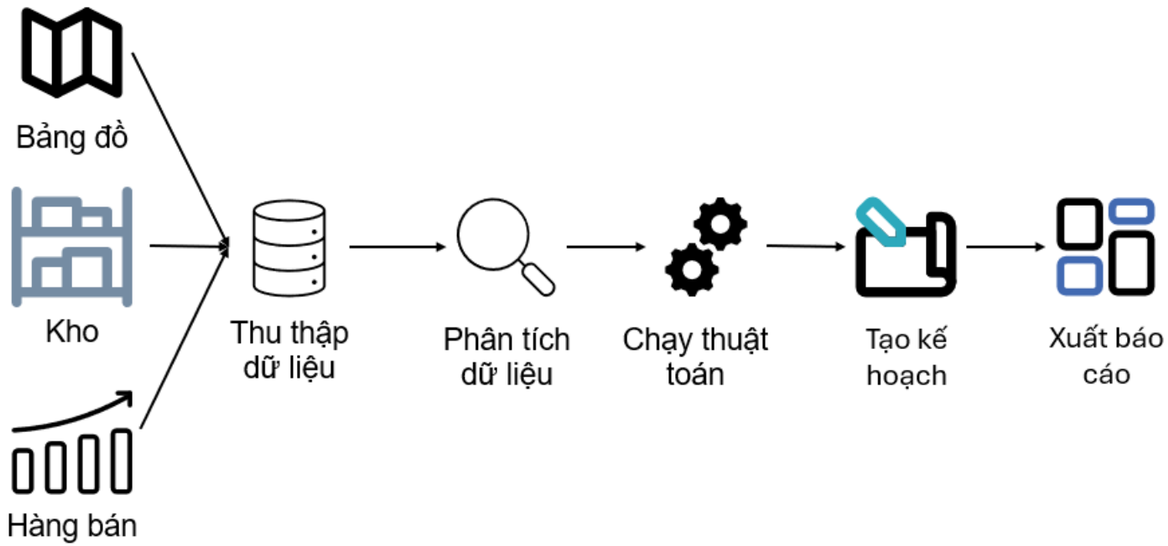
Áp dụng các thuật toán tối ưu (ví dụ: Rule-based hoặc Genetic Algorithm) để tìm ra phương án luân chuyển hàng hiệu quả nhất.

4. Tạo kế hoạch

Xây dựng kế hoạch luân chuyển hàng chi tiết, bao gồm: địa điểm nguồn, địa điểm đích, sản phẩm và số lượng cụ thể.

5. Xuất báo cáo

Tạo các báo cáo tổng hợp, hiển thị các chỉ số KPI, trực quan hóa kết quả và đưa ra các khuyến nghị tối ưu.



Các thành phần dữ liệu đầu vào

stores.csv: Thông tin cửa hàng (ID, tên, vị trí địa lý)

store_id	store_name	city	latitude	longitude
1	Lan Chi - Indochina Plaza	Hanoi	20.9	105.9
2	Hapro Thanh Xuân	Hanoi	21.0	105.8

products.csv: Danh mục sản phẩm (ID, tên, danh mục, giá)

product_id	product_name	category	price	cost
1	Dell Galaxy Buds2	Electronics	1,830,000	536,666
2	Lenovo Samsung T7 SSD	Electronics	1,080,000	243,820

sales_data.csv: Lịch sử bán hàng theo ngày

date	store_id	product_id	quantity	revenue	cost
2025-07-12	1	18	8	2,080,000	480,843
2025-07-12	1	3	6	7,440,000	220,5393

Các thành phần dữ liệu đầu vào

inventory_data.csv: Tồn kho hiện tại của từng cửa hàng

store_id	product_id	current_stock	last_updated
1	18	210	2025-10-10
2	18	163	2025-10-10

distance_matrix.cs: Ma trận khoảng cách

	1	2	…	20
1	0	19.96	…	1120.95
2	19.96	0	…	1138.37
3	14.24		0	

transport_cost_matrix.csv: Ma trận chi phí

	1	2	…	20
1	0	19.96	…	1120.95
2	19.96	0	…	1138.37
3	14.24		0	

Days of Supply: Lượng hàng hiện tại đủ bán bao nhiêu ngày?

Công thức:

Days of Supply = Tồn kho hiện tại / Lượng bán trung bình mỗi ngày

Ví dụ: *Cửa hàng có 60 chiếc áo, bán trung bình 4 chiếc/ngày. Days of Supply = 60 ÷ 4 = 15 ngày*

Thừa hàng (> 21 ngày)

Quá nhiều hàng, có thể ế ẩm, chiếm vốn lưu động.

Thiếu hàng (< 7 ngày)

Quá ít hàng, có thể hết hàng, mất cơ hội bán và khách hàng.

Lý tưởng (7-21 ngày)

Lượng hàng vừa đủ, an toàn, tối ưu cho hoạt động kinh doanh.

Ví dụ tính toán cụ thể Days of Supply

Ví dụ cụ thể về Days of supply

1. Dữ liệu đầu vào

- **Cửa hàng A**: Sản phẩm X có **100 chiếc**, bán TB **3 chiếc/ngày**
- **Cửa hàng B**: Sản phẩm X có 15 chiếc, bán TB **5 chiếc/ngày**

2. Phân tích Days of Supply

- **Cửa hàng A**: DoS = 100 / 3 = **33 ngày** → THỪA 39 chiếc (vượt quá mức lý tưởng 21 ngày)
- **Cửa hàng B**: DoS = 15 / 5 **= 3 ngày** → THIẾU 20 chiếc (dưới mức tối thiểu 7 ngày)

4. Cân nhắc và kết quả tối ưu

- Cần chuyển hàng từ **Cửa hàng A** sang **Cửa hàng B** để cân bằng tồn kho.
  - Mức tồn kho lý tưởng: 7-21 ngày
  - Số lượng cần chuyển: Ít nhất 20 chiếc để B đạt mức tối thiểu.

4. Cân nhắc và kết quả tối ưu

- **Chi phí vận chuyển**: Tối thiểu hóa chi phí.
- **Khoảng cách**: Xem xét quãng đường giữa A và B.
- **Số lượng tối ưu**: Chuyển 20 chiếc để B đạt DoS 7 ngày (15+20)/5 = 7. Sau đó A sẽ còn 80 chiếc, DoS 80/3 = 26.6 ngày. Cần chuyển thêm 17 chiếc từ A nữa để A đạt 21 ngày.

Xác định hàng thừa (Excess Inventory)

Hàng thừa là số lượng sản phẩm vượt quá mức cần thiết để bán trong tương lai

Định nghĩa

Số lượng sản phẩm vượt quá mức cần thiết để bán trong 21 ngày.

Công thức đơn giản

Hàng thừa = Tồn kho hiện tại - Lượng cần để bán 21 ngày

Lượng cần để bán 21 ngày = Bán trung bình mỗi ngày × 21

Ví dụ cụ thể

Cửa hàng có **100 chiếc áo**, bán **3 chiếc/ngày**
Lượng cần để bán 21 ngày: 3 × 21 = **63 chiếc**
Hàng thừa: 100 - 63 = **37 chiếc**

Ý nghĩa

37 chiếc hàng thừa này có thể được chuyển cho cửa hàng khác đang cần hàng để tối ưu tồn kho toàn hệ thống.

Xác định hàng thiếu (Needed Inventory)

Hàng thừa là số lượng sản phẩm vượt quá mức cần thiết để bán trong tương lai

Định nghĩa

Hàng thiếu là số lượng sản phẩm cần bổ sung để đảm bảo bán được ít nhất 7 ngày.

Công thức đơn giản

Hàng thiếu = Lượng cần để bán 7 ngày - Tồn kho hiện tại

Lượng cần để bán 7 ngày = Bán trung bình mỗi ngày × 7

Ví dụ cụ thể

Cửa hàng có **15 chiếc áo**, bán **5 chiếc/ngày**
Lượng cần tối thiểu: **5 × 7 = 35 chiếc**
Hàng thiếu: 35 - 15 = **20 chiếc**

Ý nghĩa

Cần nhận thêm 20 chiếc từ cửa hàng khác để tránh hết hàng.

Khoảng cách và chi phí vận chuyển

Để quyết định chuyển hàng từ đâu đến đâu, ta cần biết khoảng cách và chi phí giữa các cửa hàng.

Khoảng cách giữa các cửa hàng

Đo lường bằng kilômét (km)) giữa hai địa điểm.
Ví dụ: Từ cửa hàng A đến B là 15km.

Chi phí vận chuyển

Số tiền cần trả để di chuyển một sản phẩm.
Ví dụ: 1000 VND/sản phẩm cho quãng đường 15km.

Nguyên tắc

Cửa hàng càng gần nhau, chi phí vận chuyển thường càng thấp.

Chuyển từ A đến B

Khoảng cách: 15km
Chi phí: 1000 VND/sản phẩm

Chuyển từ A đến C

Khoảng cách: 50km
Chi phí: **3000 VND/sản phẩm**

Kết luận

Ưu tiên chuyển hàng **từ A đến B** vì chi phí rẻ hơn.

Kết quả mà hệ thống cung cấp

Sau khi chạy thuật toán, hệ thống sẽ đưa ra những thông tin cụ thể để thực hiện chuyển hàng.

Kế hoạch chuyển hàng chi tiết

- Danh sách cụ thể từng lần chuyển (từ cửa hàng nào, đến cửa hàng nào, sản phẩm gì, số lượng, chi phí).
- Ví dụ: Chuyển 20 chiếc áo từ cửa hàng A đến cửa hàng B, chi phí 20,000 VND.

Tổng kết số liệu

- Tổng chi phí, tổng số lần chuyển, tổng số sản phẩm được di chuyển.

Báo cáo hiệu quả

- Giảm được bao nhiêu trường hợp thừa hàng và thiếu hàng.

Tóm tắt Phần 1

Bài toán tối ưu phân bổ hàng hóa trong chuỗi bán lẻ

Khái niệm Days of Supply và cách tính

Xác định hàng thừa (Excess) và hàng thiếu (Needed)

Dữ liệu đầu vào: tồn kho, lịch sử bán hàng, khoảng cách, chi phí

Mục tiêu: tối thiểu hóa chi phí vận chuyển

Outline bài giảng

AI VIETNAM
@aivietnam.edu.vn

PHẦN 1: Giới thiệu & Phân tích bài toán

- Tổng quan về tối ưu hóa
- Phân tích vấn đề hiện tại
- Xác định mục tiêu và phạm vi

PHẦN 2: Thiết kế & Triển khai mô hình tối ưu

- Lựa chọn thuật toán và công cụ
- Xây dựng cấu trúc mô hình
- Triển khai và kiểm thử mô hình

PHẦN 3: Triển khai, Đánh giá & Ứng dụng nâng cao

- Tích hợp và vận hành hệ thống
- Đánh giá hiệu suất và tối ưu
- Mở rộng và ứng dụng trong tương lai

Tối ưu hóa là gì?

Tối ưu hóa là quá trình tìm ra cách làm TỐT NHẤT trong tất cả các cách có thể.

Giống như tìm đường đi ngắn nhất từ nhà đến trường trong tất cả các con đường có thể.



Hàm mục tiêu

Cái gì cần tối ưu (ví dụ: giảm chi phí xuống thấp nhất, tăng lợi nhuận lên cao nhất).

Trong bài toán này: Tìm cách chuyển hàng với **chi phí thấp nhất**

Biến quyết định

Những quyết định cần đưa ra (ví dụ: chuyển bao nhiêu sản phẩm, từ đâu đến đâu).

Trong bài toán này: Quyết định **chuyển bao nhiêu** từ cửa hàng nào

Ràng buộc

Những quy tắc bắt buộc phải tuân theo (ví dụ: không được chuyển nhiều hơn số hàng có sẵn).

Trong bài toán này**: Không vượt quá** tồn kho có sẵn

Tổng quan về các bài toán tối ưu trong thực tế

Tối ưu hóa có ở khắp nơi trong cuộc sống hàng ngày, từ việc chọn đường đi đến việc quản lý doanh nghiệp.

Tìm đường đi ngắn nhất	Sắp xếp thời gian biểu	Quản lý ngân sách	Vận chuyển hang hóa
• Google Maps tìm đường từ nhà đến công ty với thời gian ít nhất.	• Học sinh sắp xếp môn học để tối đa hóa điểm số với thời gian học có hạn.	• Gia đình phân bổ tiền lương để tối đa hóa hạnh phúc với ngân sách có hạn.	• Công ty giao hàng tối ưu tuyến đường để giảm chi phí nhiên liệu.

Đặc điểm chung của các bài toán tối ưu

Mục tiêu rõ ràng

Có mục tiêu cần tối thiểu hóa (ví dụ: chi phí, thời gian) hoặc tối đa hóa (ví dụ: lợi nhuận, điểm số).

Giới hạn/Ràng buộc

Luôn có những giới hạn về tài nguyên như thời gian, tiền bạc, nguyên vật liệu hoặc quy tắc phải tuân thủ.

Nhiều lựa chọn

Có nhiều cách để đạt được mục tiêu, nhưng chỉ có một hoặc vài cách là tối ưu nhất.

Kết luận: Bài toán chuyển hàng của chúng ta cũng thuộc loại: **tối thiểu chi phí vận chuyển trong giới hạn tồn kho**.

Phân loại các bài toán tối ưu

Các bài toán tối ưu được phân loại dựa trên tính chất của biến quyết định và hàm mục tiêu.

Theo tính chất biến quyết định

- **Biến liên tục:** Có thể nhận giá trị thập phân (VD: 2.5 lít xăng, 3.7 kg thịt)
- **Biến rời rạc:** Chỉ nhận giá trị nguyên (ví dụ: 5 chiếc xe, 10 người)

Theo tính chất hàm mục tiêu

- **Tuyến tính:** Quan hệ tỷ lệ thuận đơn giản (ví dụ: chi phí = số lượng × đơn giá)
- **Phi tuyến:** Quan hệ phức tạp (ví dụ: giảm giá theo số lượng mua)

Đặc điểm chung của các bài toán tối ưu

Linear Programming (LP)

Kết hợp **biến liên tục** và **hàm mục tiêu tuyến tính**.
VD: Phân bổ ngân sách quảng cáo giữa các kênh để tối đa hóa hiệu quả.

Integer Programming (IP)

Kết hợp **biến rời rạc (nguyên)** và **hàm mục tiêu tuyến tính**.
VD: Chọn dự án đầu tư (có/không) từ danh sách các dự án khả thi.

Mixed Integer Linear Programming (MILP)

Kết hợp cả **biến liên tục** và **biến rời rạc** với **hàm mục tiêu tuyến tính**.
VD: Bài toán chuyển hàng (số hàng là nguyên, chi phí vận chuyển là liên tục).

Nonlinear Programming

Sử dụng **hàm mục tiêu phi tuyến**, có thể với cả biến liên tục hoặc rời rạc.
VD: Tối ưu hóa lợi nhuận khi giá sản phẩm thay đổi phi tuyến theo số lượng bán.

Định nghĩa biến quyết định

Biến quyết định là những con số mà chúng ta cần tìm ra để giải quyết bài toán.

📌 **Biến quyết định**

$$x[i, j, p]$$

- Chuyển bao nhiêu sản phẩm **p**
- Từ cửa hàng **i**
- Đến cửa hàng **j**

Ví dụ: x[A, B, áo] = 10 => chuyển 10 chiếc áo từ cửa hàng A đến cửa hàng B

Tính chất 1: Không âm $x \geq 0$

Số lượng sản phẩm chuyển không thể là số âm. Không thể chuyển **-5** chiếc áo

Tính chất 2: Số nguyên $x \in Z$

Số lượng sản phẩm phải là số nguyên. Không thể có **2.5** chiếc áo

Tính chất 3: Ma trận 3 chiều

Một "cuốn sách" dữ liệu khổng lồ: **p** trang × **i** hàng × **j** cột
Mỗi trang = 1 sản phẩm, mỗi ô = lượng chuyển

Quy mô bài toán: **20 cửa hàng và 30 sản phẩm**
=> 20 (từ) × 20 (đến) × 30 (sản phẩm) = **12,000 quyết định** cần đưa ra
=> cần máy tính để giải quyết – quá nhiều quyết định để con người xử lý một cách hiệu quả.

Hàm mục tiêu (Objective Function)

Hàm mục tiêu xác định mục tiêu tối ưu của bài toán, thường là giảm chi phí vận chuyển xuống thấp nhất có thể.

Mục tiêu chính:

Tìm phương án chuyển hàng với tổng chi phí vận chuyển thấp nhất.

Cách tính:

Cộng chi phí của tất cả các lần chuyển hàng (giữa các cửa hàng, cho mỗi sản phẩm).

Ví dụ:

Chuyển **10** áo (A đến B) tốn **15,000 VND**.
Chuyển **5** quần (C đến D) tốn **8,000 VND**.
Tổng chi phí = **23,000 VND**.

Tại sao cần tối thiểu hóa chi phí?

Tiết kiệm cho công ty:

Trực tiếp giảm chi phí, tăng lợi nhuận và nguồn lực đầu tư.

Tối ưu hóa:

Đảm bảo hiệu quả hoạt động bằng cách ưu tiên các tuyến đường và số lượng vận chuyển tối ưu.

Máy tính thử nghiệm nhiều phương án → Tìm **tổng chi phí thấp nhất** cho toàn hệ thống

Ràng buộc về nguồn cung (Supply Constraints)

Đây là quy tắc cơ bản đảm bảo tính khả thi của bài toán tối ưu hóa vận chuyển: Một cửa hàng không thể gửi đi nhiều hàng hóa hơn số lượng hàng hóa mà nó hiện đang có sẵn.

Công thức:

Tổng số lượng sản phẩm X gửi đi từ Cửa hàng A $\leq$ Số lượng sản phẩm X có sẵn tại Cửa hàng A

$$\sum_j x[i, j, p] \leq \text{excess}[i, p]$$

Ký hiệu:

- **x[i,j,p]:** Số lượng sản phẩm p được chuyển từ cửa hàng i đến cửa hàng j.
- **needed[j,p]:** Số lượng sản phẩm p có sẵn tại cửa hàng i.
- $\sum_j$: Tổng số lượng chuyển đi từ cửa hàng i đến các cửa hàng j khác.

Ví dụ minh họa

- Cửa hàng A có 10 chiếc áo sơ mi. Cửa hàng A chỉ có thể gửi đi tối đa 10 chiếc áo sơ mi (ví dụ: 5 chiếc đến B, 3 chiếc đến C, và 2 chiếc đến D).

Ràng buộc về nhu cầu (Demand Constraints)

Tổng lượng hàng mà cửa hàng (j) nhận từ các nguồn khác (i) không được vượt quá số lượng hàng đang thiếu của sản phẩm đó

Công thức:

Tổng số lượng sản phẩm X được gửi đến Cửa hàng B $\leq$ Số lượng sản phẩm X mà Cửa hàng B đang thiếu

$$\sum_i x[i, j, p] \leq needed[j, p]$$

Ký hiệu:

- **x[i,j,p]:** Số lượng sản phẩm p được chuyển từ cửa hàng i đến cửa hàng j.
- **needed[j,p]:** Số lượng sản phẩm p mà cửa hàng j đang cần (hoặc thiếu).
- $\sum_i$: Tổng tất cả các lần chuyển hàng đến cửa hàng j từ tất cả các cửa hàng i khác.

Ví dụ minh họa

- Tình huống: Cửa hàng B đang thiếu 8 chiếc áo sơ mi màu xanh.
- Ràng buộc: Tổng số áo sơ mi màu xanh được gửi đến Cửa hàng B từ các cửa hàng khác phải $\leq$ 8 chiếc.

Ràng buộc bổ sung

Các ràng buộc này phản ánh quy tắc kinh doanh thực tế

Không tự chuyển

x[i,i,p] = 0 (không chuyển từ cửa hàng cho chính nó)

Ngưỡng tối thiểu

x[i,j,p] = 0 hoặc x[i,j,p] ≥ MIN_UNITS

Giới hạn số transfers

Mỗi cửa hàng chỉ nhận tối đa N lô hàng/ngày

Mô hình toán học

Mô hình này giúp chúng ta tìm ra cách tối ưu để chuyển hàng giữa các cửa hàng, nhằm giảm thiểu chi phí và đáp ứng nhu cầu.

1. Mục tiêu: Giảm thiểu chi phí

Tìm cách chuyển hàng giữa các cửa hàng để tổng chi phí vận chuyển là thấp nhất.

Minimize:

$$\sum_i \sum_j \sum_p \text{cost}[i,j] \times x[i,j,p]$$

- cost[i,j]: Chi phí chuyển 1 đơn vị hàng từ cửa hàng i đến cửa hàng j
- x[i,j,p]: Số lượng sản phẩm p được chuyển từ cửa hàng i đến cửa hàng j.

2. Ràng buộc cung cấp (Supply)

Lượng hàng chuyển đi từ một cửa hàng không thể vượt quá số lượng hàng hóa dư thừa tại đó.

$$\sum_j x[i,j,p] \leq \text{excess}[i,p]$$

- **excess[i,p]**: Lượng hàng hóa p đang dư thừa tại cửa hàng i.
- Ví dụ: Nếu cửa hàng A có dư 10 áo sơ mi, nó chỉ có thể chuyển đi tối đa 10 áo sơ mi.

3. Ràng buộc nhu cầu (Demand)

Lượng hàng nhận về của một cửa hàng không thể vượt quá số lượng hàng hóa đang thiếu tại đó.

$$\sum_i x[i,j,p] \leq \text{needed}[j,p]$$

- **needed[j,p]**: Lượng hàng hóa p đang cần tại cửa hàng j.
- Ví dụ: Nếu cửa hàng B thiếu 5 đôi giày, nó chỉ có thể nhận tối đa 5 đôi giày.

4. Ràng buộc về số lượng chuyển

Số lượng hàng hóa được chuyển phải là số nguyên không âm.

$$x[i,j,p] \geq 0, \text{integer } \forall\, i,j,p$$

5. Ràng buộc không tự chuyển

Một cửa hàng không thể chuyển hàng cho chính nó.

$$x[i,j,p] = 0, \forall\, i,p$$

Các phương pháp giải

Lựa chọn phương pháp phù hợp dựa trên quy mô, thời gian và chất lượng nghiệm

Phương pháp chính xác

Tìm nghiệm tối ưu toàn cục

- **Kỹ thuật**: Simplex, Branch & Bound
- **Ưu điểm**: Chính xác 100% khi tìm được nghiệm trong thời gian hợp lý
- **Nhược điểm**: Chậm, tốn tài nguyên
- **Sử dụng khi**: Bài toán nhỏ, cần độ chính xác tuyệt đối

Phương pháp Heuristic

Tìm nghiệm "đủ tốt" nhanh chóng

- **Kỹ thuật**: Greedy, Rule-based
- **Ưu điểm**: Nhanh, đơn giản
- **Nhược điểm**: Không đảm bảo tối ưu
- **Sử dụng khi**: Bài toán lớn, cần nghiệm nhanh

Phương pháp Metaheuristic

Tìm nghiệm gần tối ưu, thoát khỏi cực tiểu cục bộ

- **Kỹ thuật**: GA, Simulated Annealing
- **Ưu điểm**: Gần tối ưu cho bài toán phức tạp, quy mô lớn
- **Nhược điểm**: Phức tạp hơn, thời gian chạy lâu hơn heuristic
- **Sử dụng khi**: Bài toán quá phức tạp, cần chất lượng tốt hơn heuristic

Tiêu chí lựa chọn phương pháp

Quy mô & độ phức tạp	Thời gian tính toán	Chất lượng nghiệm yêu cầu	Khả năng triển khai & tài nguyên

Rule-Based Algorithm

Rule-based sử dụng các thông tin sẵn có và thực hiện theo ý tưởng của Greedy algorithm

Chuẩn bị dữ liệu

Sắp xếp hàng thừa theo số lượng giảm dần (ưu tiên chuyển từ nơi thừa nhiều nhất)

Sắp xếp hàng thiếu theo số lượng giảm dần (ưu tiên cung cấp cho nơi thiếu nhiều nhất)

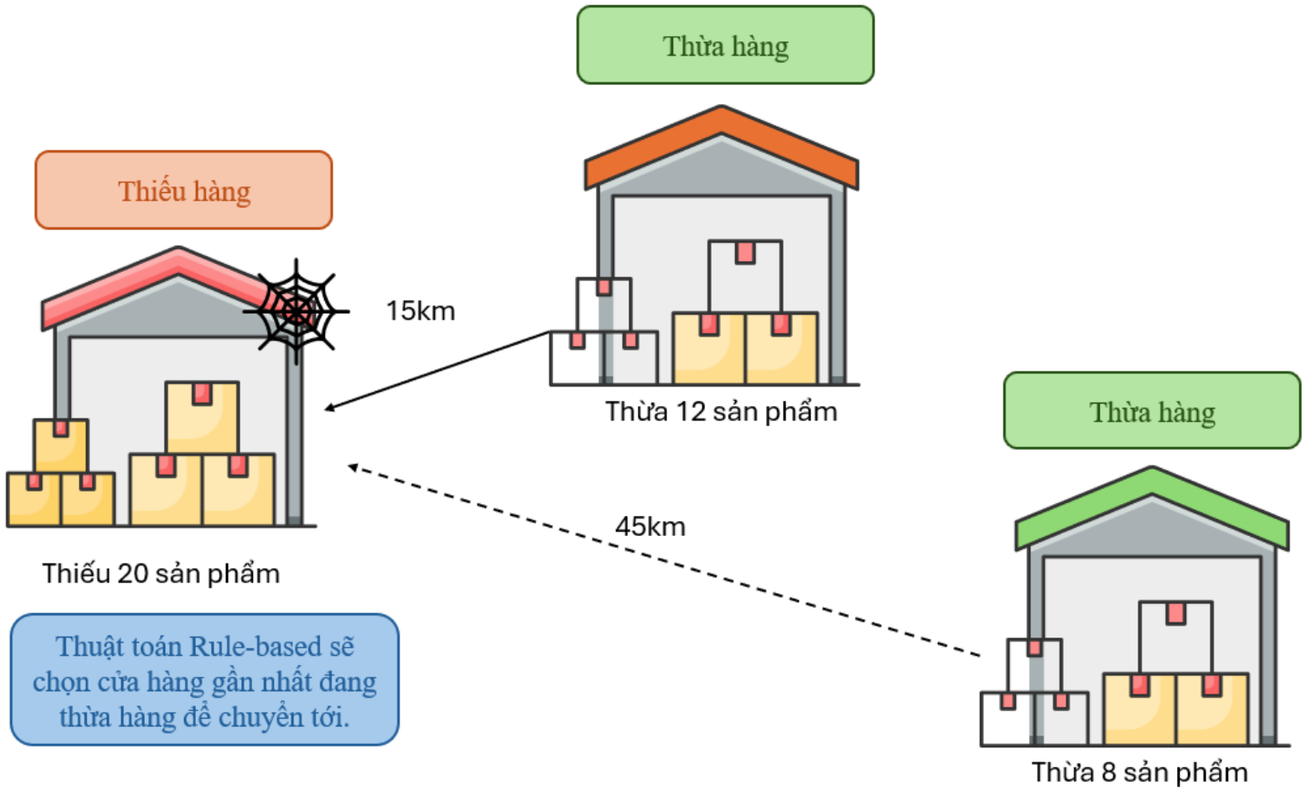
Cho mỗi mặt hàng thiếu

Tính số lượng thiếu còn lại

Tìm tất cả kho thừa cho cùng sản phẩm

Tính khoảng cách từ cửa hàng thiếu đến tất cả cửa hàng thừa

Chuyển hàng từ gần đến xa cho đến khi hết nhu cầu



Rule-Based Algorithm

Ví dụ tính toán chi tiết

Chuẩn bị dữ liệu

Sắp xếp hàng thừa theo số lượng giảm dần excess_units (giảm dần)

Sắp xếp hàng thiếu theo số lượng giảm dần needed_units (giảm dần)

Cho mỗi mặt hàng thiếu

Tính số lượng thiếu còn lại

Tìm tất cả kho thừa cho cùng sản phẩm

Tính khoảng cách từ cửa hàng thiếu đến tất cả cửa hàng thừa

Chuyển hàng từ gần đến xa cho đến khi hết nhu cầu

Dữ liệu: Điều chuyển tồn kho cho **Sản phẩm X**

- Cửa hàng A: Thiếu 20 đơn vị.
- Cửa hàng B: Thừa 8 đơn vị. Chi phí đến A: 15km, 1.500 VND/đơn vị
- Cửa hàng C: Thừa 12 đơn vị. Chi phí đến C: 45km, 2.500 VND/đơn vị.

Số lượng thiếu là 20 đơn vị

Có 2 kho thừa cho cùng sản phẩm là B và C

Ưu tiên cửa hàng B (15km) vì gần hơn cửa hàng C (45km)

Chuyển 8 đơn vị từ B đến A với chi phí là
8 đơn vị × 1.500 VND = 12.000 VND

Chuyển 12 đơn vị từ C đến A với chi phí là
12 đơn vị × 2.500 VND = 30.000 VND

Tổng Chi phí Vận chuyển: 12.000 VND + 30.000 VND = **42.000 VND**

Genetic Algorithm (Thuật toán di truyền)

Thuật toán di truyền mô phỏng tiến hóa tự nhiên để tối ưu hóa bài toán điều chuyển tồn kho, tìm ra giải pháp có tổng chi phí vận chuyển thấp nhất.

Khởi tạo quần thể: Tạo ngẫu nhiên 100-200 cá thể ban đầu. Mỗi cá thể là một kế hoạch điều chuyển tồn kho

Dùng hàm **Fitness** để đánh giá ban đầu về các cá thể

Vòng lặp tiến hóa

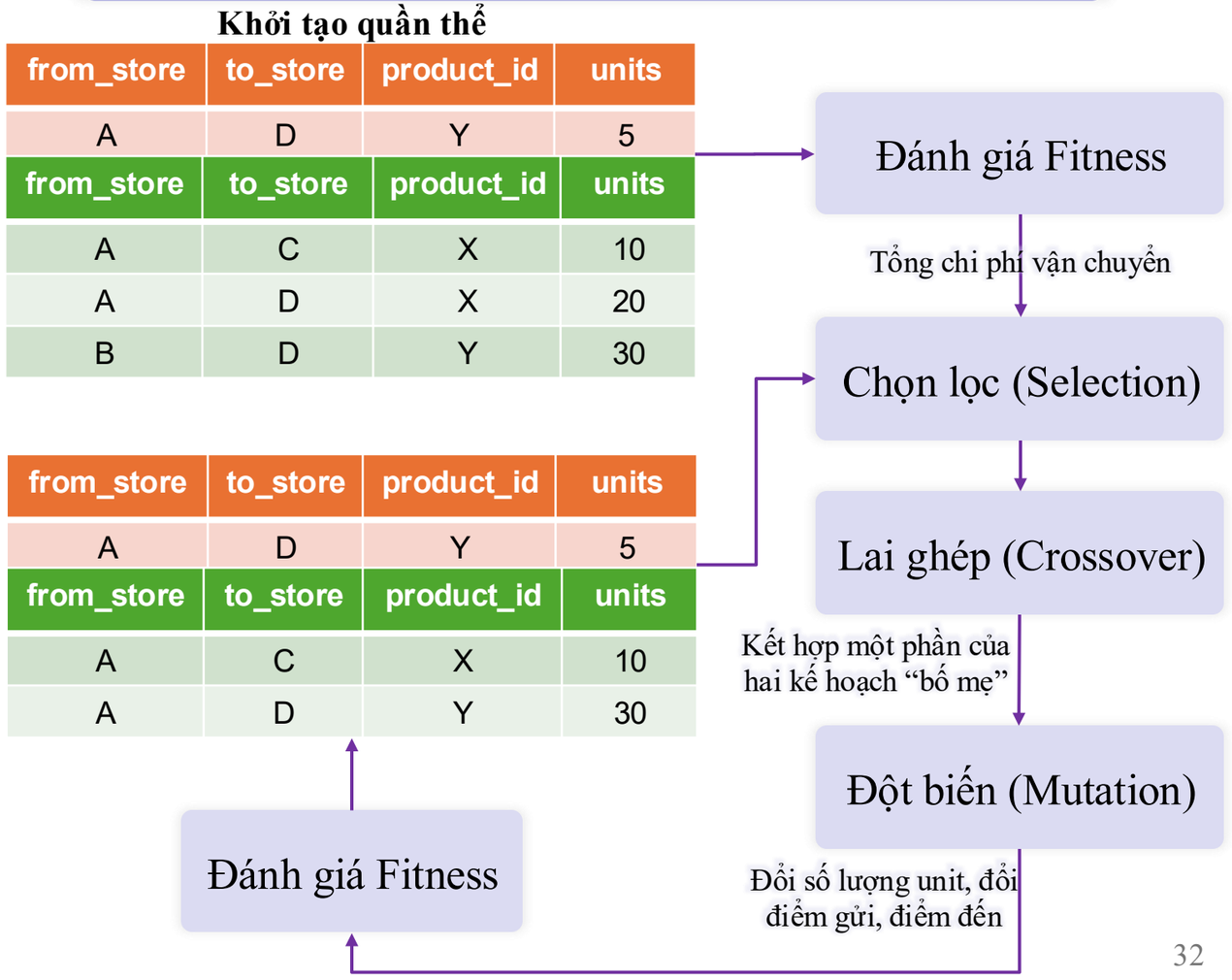
Chọn lọc (Selection): Chọn các cá thể có **fitness cao** (chi phí thấp) làm "bố mẹ" cho thế hệ sau

Lai ghép (Crossover): Kết hợp một phần của hai kế hoạch "bố mẹ" để tạo ra "con cái" mới

Đột biến (Mutation): Tạo thay đổi nhỏ, ngẫu nhiên trong kế hoạch của "con cái"

Đánh giá **Fitness** quần thể mới và tiếp tục vòng lặp với thế hệ "con cái" trở thành "bố mẹ"

Chú ý: Ở cuối mỗi vòng lặp sẽ duy trì tinh hoa (elitism) cá thể tốt nhất của "bố mẹ" hoặc "con cái"



Genetic Algorithm – Individual và Population

Trong thuật toán di truyền, population (quần thể) là tập hợp tất cả các giải pháp ứng viên cho bài toán, trong khi individual (cá thể) là một giải pháp đơn lẻ trong quần thể đó.

Cấu trúc một cá thể:

```
Individual = [
        {from_store: A, to_store: C, product_id: X, units: 12},
        {from_store: A, to_store: D, product_id: X, units: 8},
        {from_store: B, to_store: D, product_id: Y, units: 10}
        ]
```

Mỗi cá thể là một phương án chuyển hàng hoàn chỉnh

Khởi tạo một quần thể bằng cách tạo một quần thể có N cá thể ngẫu nhiên

```
Individual_1 = [
        {from_store: A, to_store: C, product_id: X, units: 12},
        {from_store: A, to_store: D, product_id: X, units: 8},
        {from_store: B, to_store: D, product_id: Y, units: 10}
        ]
```

```
Individual_2 = [
        {from_store: A, to_store: D, product_id: X, units: 1},
        {from_store: B, to_store: D, product_id: X, units: 7},
        {from_store: C, to_store: E, product_id: Y, units: 9}
        ]
```

Genetic Algorithm – Fitness và Selection

Hàm **fitness** đánh giá và cho ra giá trị độ thích nghi của mỗi cá thể. **Tournament Selection** là phương pháp chọn ngẫu nhiên k cá thể, sau đó chọn cá thể tốt nhất để sinh sản.

Tính fitness của một cá thể:

```
Individual = [
        {from_store: A, to_store: C, product_id: X, units: 12},
        {from_store: A, to_store: D, product_id: X, units: 8},
        {from_store: B, to_store: D, product_id: Y, units: 10}
        ]
```

Fitness càng thấp càng tốt (minimize cost)
*total_cost += total_cost_of_one_unit * units*

Tournament Selection:

Chọn ngẫu nhiên K cá thể (K=3-5) từ quần thể (N cá thể)

So sánh fitness, chọn cá thể tốt nhất

Đưa cá thể tốt nhất vào danh sách winner

Lặp lại cho đến khi đủ số lượng "bố mẹ"

Ưu điểm: Đơn giản, hiệu quả, duy trì đa dạng

Genetic Algorithm - Crossover (Lai ghép)

Crossover (lai ghép) là phép toán kết hợp thông tin di truyền từ hai cá thể "bố mẹ" bằng cách trao đổi các đoạn thông tin để tạo ra cá thể con mới kế thừa đặc tính tốt từ cả hai.

Single-Point Crossover (=1):

Parent1: [Transfer_A, Transfer_B, Transfer_C, Transfer_D]

Parent2: [Transfer_X, Transfer_Y, Transfer_Z]

Crossover point = 1

Child1: [Transfer_A, Transfer_Y, Transfer_Z]

Child2: [Transfer_X, Transfer_B, Transfer_C, Transfer_D]

Single-Point Crossover (=2):

Parent1: [Transfer_A, Transfer_B, Transfer_C, Transfer_D]

Parent2: [Transfer_X, Transfer_Y, Transfer_Z]

Crossover point = 2

Child1: [Transfer_A, Transfer_B, Transfer_Z]

Child2: [Transfer_X, Transfer_Y, Transfer_C, Transfer_D]

Genetic Algorithm - Mutation (Đột biến)

Mutation (đột biến) là phép toán thay đổi ngẫu nhiên giá trị tại một số vị trí trong cùng 1 cá thể để tạo sự đa dạng cho quần thể

Change quantity: Thay đổi số lượng ngẫu nhiên của **units** vận chuyển

Individual = [
{from_store: A, to_store: C, product_id: X, units: 12},
{from_store: A, to_store: D, product_id: X**, units: 8**},
{from_store: B, to_store: D, product_id: Y, units: 10}
]

from_store: A, to_store: D, product_id: X, **units: 8**
Random units có thể nhận (1-max_units)

Change quantity: Giả sử max_units = 15 và random ra 12

Individual = [
{from_store: A, to_store: D, product_id: X, units: 12},
{from_store: B, to_store: D, product_id: X, **units: 12**},
{from_store: C, to_store: D, product_id: X, units: 10}
]

Sau khi thay thế giá trị units mới.
Sẽ có một hàm chỉnh sửa lại để tuân theo các ràng buộc

Genetic Algorithm - Mutation (Đột biến)

Mutation (đột biến) là phép toán thay đổi ngẫu nhiên giá trị tại một số vị trí trong cùng 1 cá thể để tạo sự đa dạng cho quần thể

Remove transfer: Một transfer bất kỳ trong cá thể sẽ bị remove ngẫu nhiên

Individual = [
{from_store: A, to_store: C, product_id: X, units: 12},
{from_store: A, to_store: D, product_id: X, units: 8},
~~{from_store: B, to_store: D, product_id: Y, units: 10}~~
]

~~from_store: B, to_store: D, product_id: Y, units: 10~~
Chọn một transfer ngẫu nhiên và remove

Add transfer: Tạo 1 transfer ngẫu nhiên và đưa vào danh sách transfer của cá thể

Individual = [
{from_store: A, to_store: C, product_id: X, units: 12},
{from_store: A, to_store: D, product_id: X, units: 8},
{from_store: B, to_store: D, product_id: Y, units: 10}
{from_store: E, to_store: F, product_id: Y, units: 2}
]

from_store: E, to_store: F, product_id: Y, units: 2
Tạo 1 transfer ngẫu nhiên và add vào cá thể

Lưu ý: Sau khi mutation sẽ có một hàm chỉnh sửa lại để tuân theo các ráng buộc

Genetic Algorithm - Mutation (Đột biến)

Mutation (đột biến) là phép toán thay đổi ngẫu nhiên giá trị tại một số vị trí trong cùng 1 cá thể để tạo sự đa dạng cho quần thể

Change route: Có xác suất ngẫu nhiên thay đổi **from_store**

Individual = [
{from_store: A, to_store: D, product_id: X, units: 12},
{**from_store: E**, to_store: D, product_id: X, units: 10},
{from_store: C, to_store: D, product_id: X, units: 8}
]

from_store: E, to_store: D, product_id: X, units: 10
Thay đổi ngẫu nhiên from_store từ A sang E

Change route: Có xác suất ngẫu nhiên thay đổi **to_store**

Individual = [
{from_store: A, to_store: D, product_id: X, units: 12},
{from_store: E, **to_store: F**, product_id: X, units: 10},
{from_store: C, to_store: D, product_id: X, units: 8}
]

from_store: E, to_store: F, product_id: X, units: 10
Thay đổi ngẫu nhiên to_store từ D sang F

Lưu ý: Sau khi mutation sẽ có một hàm chỉnh sửa lại để tuân theo các ráng buộc

Tóm tắt Phần 2

Khái niệm Tối ưu hóa

Định nghĩa bài toán, các thành phần chính: hàm mục tiêu, biến quyết định, ràng buộc.

Mô hình toán học: mục tiêu giảm thiểu chi phí vận chuyển.

Ràng buộc quan trọng: nguồn cung và nhu cầu.

Hai Phương pháp Giải quyết

Rule-Based: Đơn giản, nhanh chóng, ưu tiên khoảng cách gần.

Genetic Algorithm: Phức tạp hơn, cho kết quả tối ưu hơn, mô phỏng quá trình tiến hóa tự nhiên.

Outline bài giảng

AI VIETNAM
@aivietnam.edu.vn

PHẦN 1: Giới thiệu & Phân tích bài toán

- Tổng quan về tối ưu hóa
- Phân tích vấn đề hiện tại
- Xác định mục tiêu và phạm vi

PHẦN 2: Thiết kế & Triển khai mô hình tối ưu

- Lựa chọn thuật toán và công cụ
- Xây dựng cấu trúc mô hình
- Triển khai và kiểm thử mô hình

PHẦN 3: Triển khai, Đánh giá & Ứng dụng nâng cao

- Tích hợp và vận hành hệ thống
- Đánh giá hiệu suất và tối ưu
- Mở rộng và ứng dụng trong tương lai

Tổng quan dự án

Triển khai, đánh giá và ứng dụng nâng cao

01

Cấu trúc dự án & Triển khai thuật toán

Tổ chức mã nguồn rõ ràng, phát triển và tích hợp các thuật toán Rule-Based và Genetic Algorithm bằng Python.

02

Đánh giá hiệu suất với KPIs

Xác định và đo lường hiệu suất dựa trên các chỉ số chính, đồng thời so sánh hiệu quả giữa hai thuật toán.

03

Trực quan hóa và báo cáo kết quả

Tạo biểu đồ và báo cáo để trình bày kết quả một cách trực quan.

04

Cải tiến & Ứng dụng AI nâng cao

Đề xuất cải thiện dự án và khám phá tiềm năng ứng dụng công nghệ AI/ML tiên tiến.

Cấu trúc code của dự án

Cấu trúc các thư mục chính

```
inventory_optimization/
│
├── data/               # Input data files
│
├── src/                # Source code
│
├── notebooks/          # Jupyter notebooks for analysis
│
├── visualizations/     # Generated charts and graphs
│
├── docs/               # Documentation
├── environment.yml     # Conda environment configuration
├── requirements.txt    # Python dependencies
├── results/            # Output files and reports
├── logs/               # Application logs
└── README.md           # This file
```

data/ - Chứa dữ liệu đầu vào: thông tin cửa hàng, sản phẩm, doanh số, tồn kho, ma trận khoảng cách và chi phí vận chuyển

```
├── data/                          # Input data files
│   ├── stores.csv                 # Store locations and information
│   ├── products.csv               # Product catalog
│   ├── sales_data.csv             # Past sales records
│   ├── inventory_data.csv         # Current inventory levels
│   ├── distance_matrix.csv        # Distance between stores
│   └── transport_cost_matrix.csv  # Transport costs
```

Các thành phần khác

src/ - Mã nguồn chính, bao gồm các module: tạo mạng lưới chuyển hàng, trực quan hóa, tạo dữ liệu mẫu, thuật toán tối ưu hóa

```
├── src/                              # Source code
│   ├── __init__.py
│   ├── config.py                     # Configuration settings
│   ├── main.py                       # Main program entry point
│   └── create_transfer_network.py # Create transfer network
visualization
│   ├── visualize_network.py          # Network visualization utilities
│   │
│   ├── data_generator/               # Sample data generation
│   │   ├── data_generator_main.py # Main data generator
│   │   ├── store_generator.py     # Generate store data
│   │   ├── product_generator.py   # Generate product data
│   │   ├── sales_generator.py     # Generate sales data
│   │   ├── inventory_generator.py # Generate inventory data
│   │   └── distance_calculator.py # Calculate distances between stores
│   │
│   ├── engine/                       # Optimization algorithms
│   │   ├── data_model.py             # Data structures and models
│   │   ├── analyzer.py               # Inventory analysis
│   │   ├── rule_based.py             # Rule-based optimization method
│   │   ├── genetic_algorithm.py      # Genetic algorithm optimization
│   │   ├── results_manager.py        # Results management and export
│   │   └── simplified_results.py     # Simplified results output
│   │
│   └── utils/                        # Utility functions
│       ├── __init__.py
│       └── logger.py                 # Logging configuration
│
├── notebooks/                         # Jupyter notebooks for analysis
│   └── 01_full_process_step_by_step.ipynb
├── visualizations/                    # Generated charts and graphs
│   ├── store_transfer_network.graphml
│   ├── graph_network.png
│   └── gephie_v01.gephi
├── docs/                              # Documentation
│   ├── logic.md                      # System logic documentation
│   └── 01_results_summary.xlsx       # Results summary template
```

Triển khai Rule-Based bằng Python

Mở rộng ý tưởng về thuật toán optimize để triển khai code

Chuẩn bị dữ liệu

Sắp xếp kho thừa theo excess_units (giảm dần)

Sắp xếp kho thiếu theo needed_units (giảm dần)

Cho mỗi mặt hàng thiếu

Tính số lượng thiếu còn lại

Tìm tất cả kho thừa cho cùng sản phẩm

Tính khoảng cách từ cửa hàng thiếu đến tất cả cửa hàng thừa

Chuyển hàng từ gần đến xa cho đến khi hết nhu cầu

IV. VÒNG LẶP OPTIMIZE (Vẫn ở trong optimize)

1. Lặp qua mỗi mặt hàng thiếu:
   a. Tính số lượng thiếu còn lại (điều chỉnh cho đã nhận)
   b. Tìm tất cả kho thừa cho cùng sản phẩm
   c. Tính khoảng cách từ cửa hàng thiếu đến tất cả cửa hàng thừa
   d. Sắp xếp cửa hàng thừa theo khoảng cách (gần nhất trước)

   2. Lặp qua mỗi mặt hàng thiếu:
      i. Bỏ qua khi tự chuyển cho chính nó
      ii. Tính số lượng thừa còn lại (điều chỉnh cho đã chuyển)
      iii. Tính số lượng chuyển = min(needed_units, excess_units)
      iv. Xác thực và lấy distance từ store thiếu đến các store thừa
      v. Tính chi phí vận chuyển = base_cost * transfer_units
      vi. Tạo đề xuất chuyển kho
      vii. Cập nhật từ điển theo dõi
      viii. Cập nhật số lượng thiếu còn lại
      ix. Thoát nếu nhu cầu được thỏa mãn hoàn toàn

Triển khai Rule-Based bằng Python

Các bước thực hiện rule-based: Logic thực hiện rule-based

I. KHỞI TẠO

1. Constructor class và khởi tạo các biến cần thiết
2. Load ma trận khoảng cách và chi phí vận chuyển
3. Khởi tạo từ điển parameter

II. KIỂM TRA ĐẦU VÀO

1. Kiểm tra xem excess_inventory và needed_inventory có rỗng không
2. Nếu rỗng, trả về kế hoạch chuyển kho rỗng

III. CHUẨN BỊ DỮ LIỆU

1. Sắp xếp kho thừa theo excess_units (giảm dần)
2. Sắp xếp kho thiếu theo needed_units (giảm dần)
3. Khởi tạo từ điển theo dõi số lượng đã chuyển

IV. VÒNG LẶP OPTIMIZE (Vẫn ở trong optimize)

1. Lặp qua mỗi mặt hàng thiếu:
   a. Tính số lượng thiếu còn lại (điều chỉnh cho đã nhận)
   b. Tìm tất cả kho thừa cho cùng sản phẩm
   c. Tính khoảng cách từ cửa hàng thiếu đến tất cả cửa hàng thừa
   d. Sắp xếp cửa hàng thừa theo khoảng cách (gần nhất trước)

   2. Lặp qua mỗi mặt hàng thiếu:
      i. Bỏ qua khi tự chuyển cho chính nó
      ii. Tính số lượng thừa còn lại (điều chỉnh cho đã chuyển)
      iii. Tính số lượng chuyển = min(needed_units, excess_units)
      iv. Xác thực và lấy distance từ store thiếu đến các store thừa
      v. Tính chi phí vận chuyển = base_cost * transfer_units
      vi. Tạo đề xuất chuyển kho
      vii. Cập nhật từ điển theo dõi
      viii. Cập nhật số lượng thiếu còn lại
      ix. Thoát nếu nhu cầu được thỏa mãn hoàn toàn

Triển khai Rule-Based bằng Python

Khởi tạo và load các data cần thiết

I. KHỞI TẠO

1. Constructor class và khởi tạo các biến cần thiết
2. Load ma trận khoảng cách và chi phí vận chuyển
3. Khởi tạo từ điển parameter

1

```
def __init__(self, distance_matrix=None, transport_cost_matrix=None):
    """
    Initialize the rule-based optimization engine.

    Args:
        distance_matrix: Matrix of distances between stores
        transport_cost_matrix: Matrix of transport costs between stores
    """
    self.distance_matrix = distance_matrix
    self.transport_cost_matrix = transport_cost_matrix
    self.transfer_plan = None
    self.logger_system = get_optimization_logger()
```

2

```
def load_matrices(self, distance_path, cost_path):

    # Load matrices with store IDs as integers
    self.distance_matrix = pd.read_csv(distance_path, index_col=0)
    self.transport_cost_matrix = pd.read_csv(cost_path, index_col=0)

    # Ensure indices and columns are integers
    self.distance_matrix.index = self.distance_matrix.index.astype(int)
    self.distance_matrix.columns = self.distance_matrix.columns.astype(int)
    self.transport_cost_matrix.index = self.transport_cost_matrix.index.astype(int)
    self.transport_cost_matrix.columns = self.transport_cost_matrix.columns.astype(
        int
    )
```

3

```
def optimize(self, excess_inventory, needed_inventory):
    # Start timing and logging
    start_time = time.time()

    parameters = {
        "excess_items": len(excess_inventory) if not excess_inventory.empty else 0,
        "needed_items": len(needed_inventory) if not needed_inventory.empty else 0,
        "algorithm": "Rule-Based Optimization",
    }

    self.logger_system.log_execution_start("rule_based_optimization", parameters)

    print("Generating rule-based transfer plan...")
    self.logger_system.log_progress(
        "rule_based_optimization", "Starting rule-based transfer plan generation..."
    )
```

Triển khai Rule-Based bằng Python

Kiểm tra điều kiện đầu vào trước khi thực hiện thuật toán

II. KIỂM TRA ĐẦU VÀO

1. Kiểm tra xem excess_inventory và needed_inventory có rỗng không
2. Nếu rỗng, trả về kế hoạch chuyển kho rỗng

```
def optimize(self, excess_inventory, needed_inventory):
        ...
        if excess_inventory.empty or needed_inventory.empty:
            message = "No excess or needed inventory found. No transfers needed."
            print(message)
            self.logger_system.log_progress("rule_based_optimization", message)
            self.transfer_plan = pd.DataFrame()

            # Log completion
            execution_time = time.time() - start_time
            results = {
                "transfers_generated": 0,
                "reason": "No excess or needed inventory",
            }
            self.logger_system.log_execution_end(
                "rule_based_optimization", execution_time, results
            )
            return self.transfer_plan
```

Triển khai Rule-Based bằng Python

III. CHUẨN BỊ DỮ LIỆU

1. Sắp xếp kho thừa theo excess_units (giảm dần)
2. Sắp xếp kho thiếu theo needed_units (giảm dần)
3. Khởi tạo từ điển theo dõi số lượng đã chuyển

```
def optimize(self, excess_inventory, needed_inventory):
    ...
    # Create a list to store transfer recommendations
    transfers = []

1   # Sort excess inventory by excess_units (descending)
    excess_sorted = excess_inventory.sort_values("excess_units", ascending=False)

2   # Sort needed inventory by needed_units (descending)
    needed_sorted = needed_inventory.sort_values("needed_units", ascending=False)

3   # Track how much has been transferred from each excess item
    transferred_from = {}
    for _, row in excess_sorted.iterrows():
        key = (row["store_id"], row["product_id"])
        transferred_from[key] = 0

    # Track how much has been transferred to each needed item
    transferred_to = {}
    for _, row in needed_sorted.iterrows():
        key = (row["store_id"], row["product_id"])
        transferred_to[key] = 0
```

Triển khai Rule-Based bằng Python

IV. VÒNG LẶP OPTIMIZE (Vẫn ở trong optimize)

1. Lặp qua mỗi mặt hàng thiếu:
   a. Tính số lượng thiếu còn lại (điều chỉnh cho đã nhận)
   b. Tìm tất cả kho thừa cho cùng sản phẩm
   c. Tính khoảng cách từ cửa hàng thiếu đến tất cả cửa hàng thừa
   d. Sắp xếp cửa hàng thừa theo khoảng cách (gần nhất trước)

   2. Lặp qua mỗi mặt hàng thiếu:
      i. Bỏ qua khi tự chuyển cho chính nó
      ii. Tính số lượng thừa còn lại (điều chỉnh cho đã chuyển)
      iii. Tính số lượng chuyển = min(needed_units, excess_units)
      iv. Xác thực và lấy distance từ store thiếu đến các store thừa
      v. Tính chi phí vận chuyển = base_cost * transfer_units
      vi. Tạo đề xuất chuyển kho
      vii. Cập nhật từ điển theo dõi
      viii. Cập nhật số lượng thiếu còn lại
      ix. Thoát nếu nhu cầu được thỏa mãn hoàn toàn

1

```
for _, need_row in pbar:
    need_store_id = need_row["store_id"]
    need_product_id = need_row["product_id"]
    needed_units = need_row["needed_units"]

    # Adjust for already received units
    need_key = (need_store_id, need_product_id)
    if need_key in transferred_to:
        needed_units - = transferred_to[need_key]

    if needed_units <= 0:
        continue

    # Find excess inventory for this product
    excess_for_product = excess_sorted[
        excess_sorted["product_id"] == need_product_id
    ]

    if excess_for_product.empty:
        continue

    # Sort excess stores by distance to the need store
    excess_for_product = excess_for_product.copy()

    # Add distance to the excess inventory
    excess_for_product["distance"] = excess_for_product["store_id"].apply(
        lambda x: (
            float(self.distance_matrix.loc[x, need_store_id])
            if x != need_store_id
            and x in self.distance_matrix.index
            and need_store_id in self.distance_matrix.columns
            else float("inf")
        )
    )

    # Sort by distance (closest first)
    excess_for_product = excess_for_product.sort_values("distance")
```

a

b

c

d

Triển khai Rule-Based bằng Python

IV. VÒNG LẶP OPTIMIZE (Vẫn ở trong optimize)

1. Lặp qua mỗi mặt hàng thiếu:
   a. Tính số lượng thiếu còn lại (điều chỉnh cho đã nhận)
   b. Tìm tất cả kho thừa cho cùng sản phẩm
   c. Tính khoảng cách từ cửa hàng thiếu đến tất cả cửa hàng thừa
   d. Sắp xếp cửa hàng thừa theo khoảng cách (gần nhất trước)

   2. Lặp qua mỗi mặt hàng thiếu:
      i. Bỏ qua khi tự chuyển cho chính nó
      ii. Tính số lượng thừa còn lại (điều chỉnh cho đã chuyển)
      iii. Tính số lượng chuyển = min(needed_units, excess_units)
      iv. Xác thực và lấy distance từ store thiếu đến các store thừa
      v. Tính chi phí vận chuyển = base_cost * transfer_units
      vi. Tạo đề xuất chuyển kho
      vii. Cập nhật từ điển theo dõi
      viii. Cập nhật số lượng thiếu còn lại
      ix. Thoát nếu nhu cầu được thỏa mãn hoàn toàn

2

```
# Transfer from closest stores with excess first
 for _, excess_row in excess_for_product.iterrows():
    excess_store_id = excess_row["store_id"]
    excess_product_id = excess_row["product_id"]
    excess_units = excess_row["excess_units"]

i   # Skip self-transfers
    if excess_store_id == need_store_id:
        continue

    # Adjust for already transferred units
    excess_key = (excess_store_id, excess_product_id)
ii  if excess_key in transferred_from:
        excess_units -= transferred_from[excess_key]

    if excess_units <= 0:
        continue

iii # Calculate units to transfer
    transfer_units = min(needed_units, excess_units)

    if transfer_units > 0:
iv      # Calculate distance from matrix
        if (
            excess_store_id in self.distance_matrix.index
            and need_store_id in self.distance_matrix.columns
        ):
            distance = float(
                self.distance_matrix.loc[excess_store_id, need_store_id]
            )
        else:
            distance = 0  # Default if distance not available
        ...
```

$$x[i,j,p] = 0, \forall\, i,p$$

$$x[i,j,p] \geq 0, \text{integer } \forall\, i,j,p$$

$$\sum_j x[i,j,p] \leq \text{excess}[i,p]$$

$$\sum_i x[i,j,p] \leq \text{needed}[j,p]$$

Triển khai Rule-Based bằng Python

IV. VÒNG LẶP OPTIMIZE (Vẫn ở trong optimize)

1. Lặp qua mỗi mặt hàng thiếu:
   a. Tính số lượng thiếu còn lại (điều chỉnh cho đã nhận)
   b. Tìm tất cả kho thừa cho cùng sản phẩm
   c. Tính khoảng cách từ cửa hàng thiếu đến tất cả cửa hàng thừa
   d. Sắp xếp cửa hàng thừa theo khoảng cách (gần nhất trước)

   2. Lặp qua mỗi mặt hàng thiếu:
      i. Bỏ qua khi tự chuyển cho chính nó
      ii. Tính số lượng thừa còn lại (điều chỉnh cho đã chuyển)
      iii. Tính số lượng chuyển = min(needed_units, excess_units)
      iv. Xác thực và lấy distance từ store thiếu đến các store thừa
      v. Tính chi phí vận chuyển = base_cost * transfer_units
      vi. Tạo đề xuất chuyển kho
      vii. Cập nhật từ điển theo dõi
      viii. Cập nhật số lượng thiếu còn lại
      ix. Thoát nếu nhu cầu được thỏa mãn hoàn toàn

$$\sum_i \sum_j \sum_p \text{cost}[i,j] \times x[i,j,p]$$

2

v

```
# Calculate transport cost from matrix
if (
    self.transport_cost_matrix is not None
    and excess_store_id in self.transport_cost_matrix.index
    and need_store_id in self.transport_cost_matrix.columns
):
    cost_value = self.transport_cost_matrix.loc[
        excess_store_id, need_store_id
    ]

    base_cost = float(cost_value)

    if np.isnan(base_cost) or base_cost <= 0:
        # Skip this transfer if cost is invalid
        continue

    transport_cost = base_cost * transfer_units

else:
    # Skip this transfer if stores not found in matrix or matrix is None
    continue
```

Triển khai Rule-Based bằng Python

IV. VÒNG LẶP OPTIMIZE (Vẫn ở trong optimize)

1. Lặp qua mỗi mặt hàng thiếu:
   a. Tính số lượng thiếu còn lại (điều chỉnh cho đã nhận)
   b. Tìm tất cả kho thừa cho cùng sản phẩm
   c. Tính khoảng cách từ cửa hàng thiếu đến tất cả cửa hàng thừa
   d. Sắp xếp cửa hàng thừa theo khoảng cách (gần nhất trước)

   2. Lặp qua mỗi mặt hàng thiếu:
      i. Bỏ qua khi tự chuyển cho chính nó
      ii. Tính số lượng thừa còn lại (điều chỉnh cho đã chuyển)
      iii. Tính số lượng chuyển = min(needed_units, excess_units)
      iv. Xác thực và lấy distance từ store thiếu đến các store thừa
      v. Tính chi phí vận chuyển = base_cost * transfer_units
      vi. Tạo đề xuất chuyển kho
      vii. Cập nhật từ điển theo dõi
      viii. Cập nhật số lượng thiếu còn lại
      ix. Thoát nếu nhu cầu được thỏa mãn hoàn toàn

2

vi

```
# Add transfer to recommendations
transfers.append(
    {
        "from_store_id": excess_store_id,
        "to_store_id": need_store_id,
        "product_id": need_product_id,
        "units": int(transfer_units),
        "distance_km": distance,
        "transport_cost": transport_cost,
    }
)
```

vii

```
# Update tracking dictionaries
if excess_key in transferred_from:
    transferred_from[excess_key] += transfer_units
else:
    transferred_from[excess_key] = transfer_units

if need_key in transferred_to:
    transferred_to[need_key] += transfer_units
else:
    transferred_to[need_key] = transfer_units
```

viii

```
# Update needed units
needed_units -= transfer_units
```

ix

```
if needed_units <= 0:
    break
```

Triển khai Genetic Algorithm bằng Python

Mở rộng ý tưởng về thuật toán optimize dùng GA để triển khai code

Khởi tạo quần thể: Tạo ngẫu nhiên 100-200 cá thể ban đầu. Mỗi cá thể là một kế hoạch điều chuyển tồn kho

Dùng hàm **Fitness** để đánh giá ban đầu về các cá thể

Vòng lặp tiến hóa

Chọn lọc (Selection): Chọn các cá thể có **fitness cao** (chi phí thấp) làm "bố mẹ" cho thế hệ sau

Lai ghép (Crossover): Kết hợp một phần của hai kế hoạch "bố mẹ" để tạo ra "con cái" mới

Đột biến (Mutation): Tạo thay đổi nhỏ, ngẫu nhiên trong kế hoạch của "con cái"

Đánh giá **Fitness** quần thể mới và tiếp tục vòng lặp với thế hệ "con cái" trở thành "bố mẹ"

Chú ý: Ở cuối mỗi vòng lặp sẽ duy trì tinh hoa (elitism) cá thể tốt nhất của "bố mẹ" hoặc "con cái"

III. KHỞI TẠO QUẦN THỂ VÀ ĐÁNH GIÁ

1. Tạo quần thể cá thể ngẫu nhiên (mỗi cá thể là một kế hoạch vận chuyển)
2. Tính fitness (tổng chi phí) cho từng cá thể
3. Chọn cá thể tốt nhất (dùng sau này cho duy trì tinh hoa)

IV. VÒNG LẶP OPTIMIZE (TIẾN HÓA)

1. Chọn lọc (tournament) để lấy danh sách bố mẹ
2. Lai ghép (có xác suất), sau đó chỉnh sửa vi phạm ràng buộc
3. Đột biến (có xác suất), sau đó chỉnh sửa vi phạm ràng buộc
4. Cắt tỉa để duy trì kích thước quần thể ban đầu
5. Đánh giá quần thể mới
6. Duy trì tinh hoa (elitism) cá thể tốt nhất so với "bố mẹ" và "con cái" sẽ được giữ lại. Sau đó thay thế quần thể

Triển khai Genetic Algorithm bằng Python

AI VIETNAM
@aivietnam.edu.vn

Các bước thực hiện GA: Logic thực hiện GA

I. KHỞI TẠO

1. Constructor class và khởi tạo các biến cần thiết
2. Load ma trận khoảng cách và chi phí vận chuyển
3. Lấy các config cho GA

II. KIỂM TRA ĐẦU VÀO & CHUẨN BỊ DATA

1. Kiểm tra xem excess_inventory và needed_inventory có rỗng không. Nếu rỗng, trả về kế hoạch chuyển kho rỗng
2. Xác định tập sản phẩm hợp lệ (vừa thừa vừa thiếu)

III. KHỞI TẠO QUẦN THỂ VÀ ĐÁNH GIÁ

1. Tạo quần thể cá thể ngẫu nhiên (mỗi cá thể là một kế hoạch vận chuyển)
2. Tính fitness (tổng chi phí) cho từng cá thể
3. Chọn cá thể tốt nhất (dùng sau này cho duy trì tinh hoa)

IV. VÒNG LẶP OPTIMIZE (TIẾN HÓA)

1. Chọn lọc (tournament) để lấy danh sách bố mẹ
2. Lai ghép (có xác suất), sau đó chỉnh sửa vi phạm ràng buộc
3. Đột biến (có xác suất), sau đó chỉnh sửa vi phạm ràng buộc
4. Cắt tỉa để duy trì kích thước quần thể ban đầu
5. Đánh giá quần thể mới
6. Duy trì tinh hoa (elitism) cá thể tốt nhất so với “bố mẹ” và “con cái” sẽ được giữ lại. Sau đó thay thế quần thể

Triển khai Genetic Algorithm bằng Python

Khởi tạo và load các data cần thiết

I. KHỞI TẠO

1. Constructor class và khởi tạo các biến cần thiết
2. Load ma trận khoảng cách và chi phí vận chuyển
3. Lấy các config cho GA

1

```
def __init__( self, distance_matrix, transport_cost_matrix, random_seed):
        self.distance_matrix = distance_matrix
        self.transport_cost_matrix = transport_cost_matrix
        self.transfer_plan = None
        self.random_seed = random_seed or RANDOM_SEED
        self.best_solution = None
        self.best_fitness = None
        self.logger_system = get_optimization_logger()

        # Set random seed
        random.seed(self.random_seed)
        np.random.seed(self.random_seed)
```

2

```
def load_matrices(self, distance_path, cost_path):

        # Load matrices with store IDs as integers
        self.distance_matrix = pd.read_csv(distance_path, index_col=0)
        self.transport_cost_matrix = pd.read_csv(cost_path, index_col=0)

        # Ensure indices and columns are integers
        self.distance_matrix.index = self.distance_matrix.index.astype(int)
        self.distance_matrix.columns = self.distance_matrix.columns.astype(int)
        self.transport_cost_matrix.index = self.transport_cost_matrix.index.astype(int)
        self.transport_cost_matrix.columns = self.transport_cost_matrix.columns.astype(
            int
        )
```

3

```
def optimize(self, excess_inventory, needed_inventory, population_size,
num_generations, crossover_prob, mutation_prob, tournament_size, verbose,):
        # Get config defaults
        config = get_ga_config()
        population_size = population_size or config["population_size"]
        num_generations = num_generations or config["num_generations"]
        crossover_prob = crossover_prob or config["crossover_prob"]
        mutation_prob = mutation_prob or config["mutation_prob"]
        tournament_size = tournament_size or config["tournament_size"]
        ...
```

Triển khai Genetic Algorithm bằng Python

AI VIETNAM
@aivietnam.edu.vn

Kiểm tra điều kiện đầu vào và xác định các sản phẩm hợp lệ trước khi thực hiện thuật toán

II. KIỂM TRA ĐẦU VÀO

1. Kiểm tra xem excess_inventory và needed_inventory có rỗng không. Nếu rỗng, trả về kế hoạch chuyển kho rỗng
2. Xác định tập sản phẩm hợp lệ (vừa thừa vừa thiếu)

1

```
def optimize(self, excess_inventory, needed_inventory):
    ...
     # Validate inputs
    if excess_inventory.empty or needed_inventory.empty:
        message = "No excess or needed inventory found. No transfers needed."
        print(message)
        self.logger_system.log_progress("genetic_algorithm_optimization", message)
        self.transfer_plan = pd.DataFrame()

        # Log completion
        execution_time = time.time() - start_time
        results = {
            "transfers_generated": 0,
            "reason": "No excess or needed inventory",
        }
        self.logger_system.log_execution_end(
            "genetic_algorithm_optimization", execution_time, results
        )
        return self.transfer_plan
```

2

```
def optimize(self, excess_inventory, needed_inventory):
    ...
     # Store data for GA operations
    self.excess_inventory = excess_inventory
    self.needed_inventory = needed_inventory

    # Find products that have both excess and shortage
    excess_products = set(excess_inventory["product_id"].unique())
    needed_products = set(needed_inventory["product_id"].unique())
    self.valid_products = list(excess_products & needed_products)

    if not self.valid_products:
        print("No products with both excess and shortage found.")
        self.transfer_plan = pd.DataFrame()
        return self.transfer_plan
```

Triển khai Genetic Algorithm bằng Python

Kiểm tra điều kiện đầu vào trước khi thực hiện thuật toán

III. KHỞI TẠO QUẦN THỂ VÀ ĐÁNH GIÁ

1. Tạo quần thể cá thể ngẫu nhiên (mỗi cá thể là một kế hoạch vận chuyển)
2. Tính fitness (tổng chi phí) cho từng cá thể
3. Chọn cá thể tốt nhất (dùng sau này cho duy trì tinh hoa)

```
def optimize(self, excess_inventory, needed_inventory):
    ...
    #  Initialize Population
    print("\nStep 1: Creating initial population...")
    self.logger_system.log_progress(
        "genetic_algorithm_optimization", "Step 1: Creating initial population..."
    )
    population = self._create_initial_population(population_size)

    # Evaluate initial population
    print("Step 2: Evaluating initial fitness...")
    self._evaluate_population(population)

    best_individual = min(population, key=lambda x: x.fitness)
```

1
2
3

Triển khai Genetic Algorithm bằng Python

Thực hiện các bước selection, lai ghép, và đột biến để thu thập được các cá thể "con cái" mới

IV. VÒNG LẶP OPTIMIZE (TIẾN HÓA) (Vẫn ở trong optimize)

1. Chọn lọc (tournament) để lấy danh sách bố mẹ
2. Lai ghép (có xác suất), sau đó chỉnh sửa vi phạm ràng buộc
3. Đột biến (có xác suất), sau đó chỉnh sửa vi phạm ràng buộc
4. Cắt tỉa để duy trì kích thuớc quần thể ban đầu
5. Đánh giá quần thể mới
6. Duy trì tinh hoa (elitism) cá thể tốt nhất so với "bố mẹ" và "con cái" sẽ được giữ lại. Sau đó thay thế quần thể

```
for generation in pbar:
    # Selection: Choose parents for reproduction
    parents = self._selection(population, tournament_size)

    # Create new generation
    offspring = []

    # Crossover: Create children by combining parents
    for i in range(0, len(parents), 2):
        parent1 = parents[i]
        parent2 = parents[i + 1] if i + 1 < len(parents) else parents[0]

        if random.random() < crossover_prob:
            child1, child2 = self._crossover(parent1, parent2)
        else:
            child1, child2 = parent1.copy(), parent2.copy()

        offspring.extend([child1, child2])

    # Mutation: Random changes to maintain diversity
    for individual in offspring:
        if random.random() < mutation_prob:
            self._mutate(individual)
```

1 2 3

Triển khai Genetic Algorithm bằng Python

Duy trì kích thước quần thể ban đầu, đánh giá và duy trì tinh hoa trong quần thể

IV. VÒNG LẶP OPTIMIZE (TIẾN HÓA) (Vẫn ở trong optimize)

1. Chọn lọc (tournament) để lấy danh sách bố mẹ
2. Lai ghép (có xác suất), sau đó chỉnh sửa vi phạm ràng buộc
3. Đột biến (có xác suất), sau đó chỉnh sửa vi phạm ràng buộc
4. Cắt tỉa để duy trì kích thước quần thể ban đầu
5. Đánh giá quần thể mới
6. Duy trì tinh hoa (elitism) cá thể tốt nhất so với "bố mẹ" và "con cái" sẽ được giữ lại. Sau đó thay thế quần thể

4

```
...
# Ensure population size
offspring = offspring[:population_size]
```

5

```
# Evaluate new generation
self._evaluate_population(offspring)
```

6

```
# Elitism: Keep the best individual from previous generation
current_best = min(offspring, key=lambda x: x.fitness)
if best_individual.fitness < current_best.fitness:
    # Replace worst individual with previous best
    worst_idx = max(
        range(len(offspring)), key=lambda i: offspring[i].fitness
    )
    offspring[worst_idx] = best_individual
else:
    best_individual = current_best

# Replace population
population = offspring
```

Triển khai Genetic Algorithm bằng Python

Chi tiết cách khởi cá thể và tạo quần thể

Khởi tạo quần thể _create_initial_population

1. Khởi tạo một các thể ngẫu nhiên dùng hàm _create_random_solution()
2. Lặp lại N lần (số lượng quần thể mong muốn)

```
def _create_initial_population(self, population_size):
    """
    Create the initial population of random solutions.
    Each individual represents a possible transfer plan.
    """
    population = []

    for _ in range(population_size):
        individual = Individual()
        individual.transfer_plan = self._create_random_solution()
        population.append(individual)

    return population
```

Triển khai Genetic Algorithm bằng Python

Chi tiết cách khởi cá thể và tạo quần thể

Khởi tạo cá thể _create_random_solution

Khởi tạo transfer_plan = []

1. Lặp qua các product_id trong valid_products:
   a. Lấy excess_stores và needed_stores theo product_id
   b. Nếu không lấy được danh sách cửa hàng thì bỏ qua product_id này
   c. Tạo từ điển theo dõi số lượng hàng thừa và thiếu cho từng store
   d. Trộn ngẫu nhiên thứ tự các cửa hàng để tạo sự đa dạng

   2. Lặp qua cửa hàng thiếu trong danh sách vừa trộn ngẫu nhiên:
      a. Bỏ qua nếu số lượng hàng cần <= 0
      Lặp qua cửa hàng thừa trong danh sách vừa trộn ngẫu nhiên:
         b. Bỏ qua khi chuyển hàng nội bộ
         c. Tính số lượng chuyển = min(needed_units, excess_units)
         d. Chọn ngẫu nhiên số lượng chuyển (từ 1 đến max)
         e. Tạo đề xuất chuyển kho
         f. Cập nhật số lượng còn lại
         g. Thoát nếu nhu cầu được thỏa mãn hoàn toàn

1

```
def _create_random_solution(self):
    transfer_plan = []

    # For each product that has both excess and shortage
    for product_id in self.valid_products:
        # Get stores with excess of this product
        excess_stores = self.excess_inventory[
            self.excess_inventory["product_id"] == product_id
        ].copy()

        # Get stores needing this product
        needed_stores = self.needed_inventory[
            self.needed_inventory["product_id"] == product_id
        ].copy()

        if excess_stores.empty or needed_stores.empty:
            continue

        # Track remaining units available
        excess_remaining = dict(
            zip(excess_stores["store_id"], excess_stores["excess_units"])
        )
        needed_remaining = dict(
            zip(needed_stores["store_id"], needed_stores["needed_units"])
        )

        # Randomly shuffle order to create variety
        excess_list = list(excess_remaining.items())
        needed_list = list(needed_remaining.items())
        random.shuffle(excess_list)
        random.shuffle(needed_list)
```

a

b

c

d

Triển khai Genetic Algorithm bằng Python

Chi tiết cách khởi cá thể và tạo quần thể

Khởi tạo cá thể _create_random_solution

Khởi tạo transfer_plan = []

1. Lặp qua các product_id trong valid_products:
   a. Lấy excess_stores và needed_stores theo product_id
   b. Nếu không lấy được danh sách cửa hàng thì bỏ qua product_id này
   c. Tạo từ điển theo dõi số lượng hàng thừa và thiếu cho từng store
   d. Trộn ngẫu nhiên thứ tự các cửa hàng để tạo sự đa dạng

2. Lặp qua cửa hàng thiếu trong danh sách vừa trộn ngẫu nhiên:
   a. Bỏ qua nếu số lượng hàng cần <= 0

   Lặp qua cửa hàng thừa trong danh sách vừa trộn ngẫu nhiên:
   b. Bỏ qua khi chuyển hàng nội bộ
   c. Tính số lượng chuyển = min(needed_units, excess_units)
   d. Chọn ngẫu nhiên số lượng chuyển (từ 1 đến max)
   e. Tạo đề xuất chuyển kho
   f. Cập nhật số lượng còn lại
   g. Thoát nếu nhu cầu được thỏa mãn hoàn toàn

2

```
# Make random transfers
for need_store, need_amount in needed_list:
    if need_amount <= 0:                                          # a
        continue

    for excess_store, excess_amount in excess_list:
        if excess_store == need_store or excess_amount <= 0:      # b
            continue

        # Transfer random amount (but not more than available or needed)   # c
        max_transfer = min(excess_amount, need_amount)
        if max_transfer > 0:                                      # d
            # Sometimes transfer full amount, sometimes partial
            transfer_amount = random.randint(1, max_transfer)

            transfer_plan.append(                                 # e
                {
                    "from_store": excess_store,
                    "to_store": need_store,
                    "product_id": product_id,
                    "units": transfer_amount,
                }
            )

            # Update remaining amounts                            # f
            excess_remaining[excess_store] -= transfer_amount
            needed_remaining[need_store] -= transfer_amount
            need_amount -= transfer_amount

            # Update the list item too
            for i, (store, amount) in enumerate(excess_list):
                if store == excess_store:
                    excess_list[i] = (store, excess_remaining[excess_store])
                    break

            if need_amount <= 0:                                  # d
                break
return transfer_plan
```

$$x[i,j,p] \geq 0, \text{integer } \forall i,j,p$$

$$x[i,j,p] = 0, \forall i,p$$

$$\sum_j x[i,j,p] \leq \text{excess}[i,p]$$

$$\sum_i x[i,j,p] \leq \text{needed}[j,p]$$

Triển khai Genetic Algorithm bằng Python

Chi tiết cách khởi cá thể và tạo quần thể

1. Đánh giá quần thể _evaluate_population

Lặp qua từng individual trong population:
Tính fitness và gán vào cá thể đó

2. Đánh giá cá thể _calculate_fitness

Khởi tạo total_cost = 0

Lặp qua mỗi transfer trong transfer_plan:

a. Lấy thông tin from_store, to_store, units

b. Nếu có cost trong transport_cost_matrix:
Lấy cost tính chi phí và cộng vào tổng chi phí

c. Nếu không có cost nhưng có distance trong distance_matrix:
Lấy distance tính chi phí theo base cost (1000) rồi cộng vào tổng chi phí

d. Nếu không có dữ liệu gì:
Áp dụng phạt nặng để tránh giải pháp không hợp lệ

1

```
def _evaluate_population(self, population):
    for individual in population:
        individual.fitness = self._calculate_fitness(individual.transfer_plan)
```

2

```
def _calculate_fitness(self, transfer_plan):
    total_cost = 0

    for transfer in transfer_plan:
        from_store = transfer["from_store"]
        to_store = transfer["to_store"]
        units = transfer["units"]

        # Get transport cost per unit
        if (self.transport_cost_matrix is not None
            and from_store in self.transport_cost_matrix.index
            and to_store in self.transport_cost_matrix.columns):

            cost_per_unit = float(
                self.transport_cost_matrix.loc[from_store, to_store]
            )
            total_cost += cost_per_unit * units

        elif (self.distance_matrix is not None
            and from_store in self.distance_matrix.index
            and to_store in self.distance_matrix.columns):

            # Fallback: use distance × base rate
            distance = float(self.distance_matrix.loc[from_store, to_store])
            base_cost_per_km = 1000  # Simple cost model
            total_cost += distance * base_cost_per_km * units

        else:
            # If no cost data, penalize heavily
            total_cost += 999999 * units

    return total_cost
```

a

b

c

d

Triển khai Genetic Algorithm bằng Python

Chi tiết cách khởi cá thể và tạo quần thể

1. _selection (tournament selection)

Lặp số lần bằng kích thước quần thể để chọn đủ parents:

a. Lấy ngẫu nhiên một nhóm tournament_size cá thể (hoặc ít hơn nếu quần thể nhỏ hơn tournament_size)

b. Chọn cá thể tốt nhất (fitness thấp nhất) trong nhóm và copy vào parents

2. _crossover (single-point + sửa ràng buộc)

a. Tạo hai cá thể con (child1, child2) rỗng

b. Lấy transfer plan từ parents

c. Nếu parent1 rỗng

child1 nhận plan của parent2, child2 là rỗng

d. Nếu parent 2 rỗng

child1 nhận plan của parent1, child2 là rỗng

e. Nếu cả 2 parent có plan

Chọn ngẫu nhiên single-point crossover (từ 1 đến tối thiểu của plan1 và plan2)

child1 = phần đầu parent1 + phần sau parent2

child2 = phần đầu parent2 + phần sau parent1

Chỉnh sửa lại để child1 và child2 tuân theo các ráng buộc

1

```
def _selection(self, population, tournament_size):
    parents = []
    for _ in range(len(population)):
        # Tournament: pick random individuals
        tournament = random.sample(
            population, min(tournament_size, len(population)))

        # Winner: best fitness (lowest cost)
        winner = min(tournament, key=lambda x: x.fitness)
        parents.append(winner.copy())
    return parents
```

a

b

2

```
def _crossover(self, parent1, parent2):
    child1 = Individual()
    child2 = Individual()

    plan1 = parent1.transfer_plan
    plan2 = parent2.transfer_plan

    if len(plan1) == 0:
        child1.transfer_plan = plan2.copy()
        child2.transfer_plan = []
    elif len(plan2) == 0:
        child1.transfer_plan = plan1.copy()
        child2.transfer_plan = []
    else:
        # Single-point crossover
        crossover_point = random.randint(1, min(len(plan1), len(plan2)) - 1)

        child1.transfer_plan = plan1[:crossover_point] + plan2[crossover_point:]
        child2.transfer_plan = plan2[:crossover_point] + plan1[crossover_point:]

        # Fix any constraint violations
        child1.transfer_plan = self._repair_solution(child1.transfer_plan)
        child2.transfer_plan = self._repair_solution(child2.transfer_plan)
    return child1, child2
```

a

b

c

d

e

Triển khai Genetic Algorithm bằng Python

Chi tiết cách khởi cá thể và tạo quần thể

_mutate (nhiều kiểu đột biến + sửa ràng buộc)

1. Nếu transfer plan rỗng, tạo kế hoạch ngẫu và return
Chọn ngẫu nhiên kiểu đột biến (1..4):
2. Thay đổi số units của 1 transfer:
   Tính max_units có thể, (>0) đặt units = random[1..max_units]
3. Xóa một transfer ngẫu nhiên (nếu số lượng transfer >1)
4. Thêm một transfer ngẫu nhiên hợp lệ (nếu tạo được)
5. Đổi nguồn (from_store) hoặc đích (to_store) cho 1 transfer:
   Chọn nguồn/đích mới hợp lệ theo product
6. Sau cùng: REPAIR kế hoạch để đảm bảo ràng buộc

```
def _mutate(self, individual):
    if len(individual.transfer_plan) == 0:
        # If empty, create a small random solution
        individual.transfer_plan = self._create_random_solution()
        return

    # Choose mutation type randomly
    mutation_type = random.randint(1, 4)

    if mutation_type == 1:
        # Change quantity of a random transfer
        transfer_idx = random.randint(0, len(individual.transfer_plan) - 1)
        transfer = individual.transfer_plan[transfer_idx]

        # Get max possible transfer for this route
        max_units = self._get_max_transfer(
            transfer["from_store"], transfer["to_store"], transfer["product_id"]
        )

        if max_units > 0:
            individual.transfer_plan[transfer_idx]["units"] = random.randint(
                1, max_units
            )

    elif mutation_type == 2:
        # Remove a random transfer
        if len(individual.transfer_plan) > 1:
            transfer_idx = random.randint(0, len(individual.transfer_plan) - 1)
            individual.transfer_plan.pop(transfer_idx)
```

1

2

3

Triển khai Genetic Algorithm bằng Python

Chi tiết cách khởi cá thể và tạo quần thể

_mutate (nhiều kiểu đột biến + sửa ràng buộc)

1. Nếu transfer plan rỗng, tạo kế hoạch ngẫu và return

Chọn ngẫu nhiên kiểu đột biến (1..4):

2. Thay đổi số units của 1 transfer:
   Tính max_units có thể, (>0) đặt units = random[1..max_units]
3. Xóa một transfer ngẫu nhiên (nếu số lượng transfer >1)
4. Thêm một transfer ngẫu nhiên hợp lệ (nếu tạo được)
5. Đổi nguồn (from_store) hoặc đích (to_store) cho 1 transfer:
   Chọn nguồn/đích mới hợp lệ theo product
6. Sau cùng: REPAIR kế hoạch để đảm bảo ràng buộc

4

```
elif mutation_type == 3:
    # Add a new random transfer
    new_transfer = self._create_random_transfer()
    if new_transfer:
        individual.transfer_plan.append(new_transfer)
```

5

```
else:
    # Change source or destination of a random transfer
    transfer_idx = random.randint(0, len(individual.transfer_plan) - 1)
    transfer = individual.transfer_plan[transfer_idx]

    if random.random() < 0.5:
        # Change source store
        excess_options = self.excess_inventory[
            (self.excess_inventory["product_id"] == transfer["product_id"])
            & (self.excess_inventory["store_id"] != transfer["from_store"])]
        if len(excess_options) > 0:
            new_source = random.choice(excess_options["store_id"].tolist())
            individual.transfer_plan[transfer_idx]["from_store"] = new_source
    else:
        # Change destination store
        needed_options = self.needed_inventory[
            (self.needed_inventory["product_id"] == transfer["product_id"])
            & (self.needed_inventory["store_id"] != transfer["to_store"])
        ]
        if len(needed_options) > 0:
            new_dest = random.choice(needed_options["store_id"].tolist())
            individual.transfer_plan[transfer_idx]["to_store"] = new_dest
```

6

```
# Fix any constraint violations after mutation
individual.transfer_plan = self._repair_solution(individual.transfer_plan)
```

Triển khai Genetic Algorithm bằng Python

Chi tiết cách khởi cá thể và tạo quần thể

_repair_solution (sửa vi phạm ràng buộc)

1. Nếu transfer plan rỗng, trả về rỗng
2. Khởi tạo từ điển theo dõi và danh sách transfer hợp lệ

Lặp qua mỗi transfer trong transfer plan:

3. Lấy thông tin from_store, to_store, product_id, units
4. Bỏ qua khi chuyển hàng nội bộ

4. Thêm một transfer ngẫu nhiên hợp lệ (nếu tạo được)
5. Tìm số lượng thừa tối đa max_excess bằng cách tra cứu trong excess_inventory nếu tìm thấy thì lấy excess_units của row đầu tiên, không thì gán bằng 0
6. Tìm số lượng thiếu tối đa max_needed bằng cách tra cứu trong needed_inventory nếu tìm thấy thì lấy needed_units của row đầu tiên không thì gán bằng 0
7. Tính số lượng còn lại có thể sử dụng max excess/needed – used/filled
8. Nếu còn cả excess và needed:

Tính số lượng thực tế có thể chuyển = min(số muốn chuyển, còn thừa, còn cần)

Nếu số lượng > 0 Tạo lệnh chuyển hợp lệ và thêm vào danh sách hợp lệ. Sau đó cập nhật từ điển theo dõi

```
def _repair_solution(self, transfer_plan):
    if not transfer_plan:
        return []

    # Track how much is being transferred from/to each store-product
combination
    excess_used = {}  # (store_id, product_id) -> units_used
    needed_filled = {}  # (store_id, product_id) -> units_filled

    valid_transfers = []

    for transfer in transfer_plan:
        from_store = transfer["from_store"]
        to_store = transfer["to_store"]
        product_id = transfer["product_id"]
        units = transfer["units"]

        # Skip self-transfers
        if from_store == to_store:
            continue

        # Get available excess and need
        excess_key = (from_store, product_id)
        needed_key = (to_store, product_id)

        # Find max available excess
        max_excess = 0
        excess_match = self.excess_inventory[
            (self.excess_inventory["store_id"] == from_store)
            & (self.excess_inventory["product_id"] == product_id)
        ]
        if len(excess_match) > 0:
            max_excess = excess_match.iloc[0]["excess_units"]
```

Triển khai Genetic Algorithm bằng Python

Chi tiết cách khởi cá thể và tạo quần thể

_repair_solution (sửa vi phạm ràng buộc)

1. Nếu transfer plan rỗng, trả về rỗng
2. Khởi tạo từ điển theo dõi và danh sách transfer hợp lệ

Lặp qua mỗi transfer trong transfer plan:

3. Lấy thông tin from_store, to_store, product_id, units
4. Bỏ qua khi chuyển hàng nội bộ

4. Thêm một transfer ngẫu nhiên hợp lệ (nếu tạo được)
5. Tìm số lượng thừa tối đa max_excess bằng cách tra cứu trong excess_inventory nếu tìm thấy thì lấy excess_units của row đầu tiên, không thì gán bằng 0
6. Tìm số lượng thiếu tối đa max_needed bằng cách tra cứu trong needed_inventory nếu tìm thấy thì lấy needed_units của row đầu tiên không thì gán bằng 0
7. Tính số lượng còn lại có thể sử dụng max excess/needed – used/filled
8. Nếu còn cả excess và needed:

Tính số lượng thực tế có thể chuyển = min(số muốn chuyển, còn thừa, còn cần)

Nếu số lượng > 0 Tạo lệnh chuyển hợp lệ và thêm vào danh sách hợp lệ. Sau đó cập nhật từ điển theo dõi

6

```
# Find max needed
max_needed = 0
needed_match = self.needed_inventory[
    (self.needed_inventory["store_id"] == to_store)
    & (self.needed_inventory["product_id"] == product_id)
]
if len(needed_match) > 0:
    max_needed = needed_match.iloc[0]["needed_units"]
```

7

```
# Check available capacity
excess_already_used = excess_used.get(excess_key, 0)
needed_already_filled = needed_filled.get(needed_key, 0)

remaining_excess = max_excess - excess_already_used
remaining_needed = max_needed - needed_already_filled
```

8

```
if remaining_excess > 0 and remaining_needed > 0:
    # Limit transfer to what's actually possible
    actual_units = min(units, remaining_excess, remaining_needed)

    if actual_units > 0:
        valid_transfers.append(
            {
                "from_store": from_store,
                "to_store": to_store,
                "product_id": product_id,
                "units": actual_units,
            }
        )

        # Update tracking
        excess_used[excess_key] = excess_already_used + actual_units
        needed_filled[needed_key] = needed_already_filled + actual_units

return valid_transfers
```

Chạy hệ thống - Command Line

Các command options liên quan đến chạy project

❖ Sinh dữ liệu mẫu

```
python src/main.py --generate-data
```

❖ Chạy Rule-Based

```
python src/main.py --rule-based
```

❖ Chạy Genetic Algorithm

```
python src/main.py --ga
```

❖ Chạy cả 2 và so sánh

```
python src/main.py --all
```

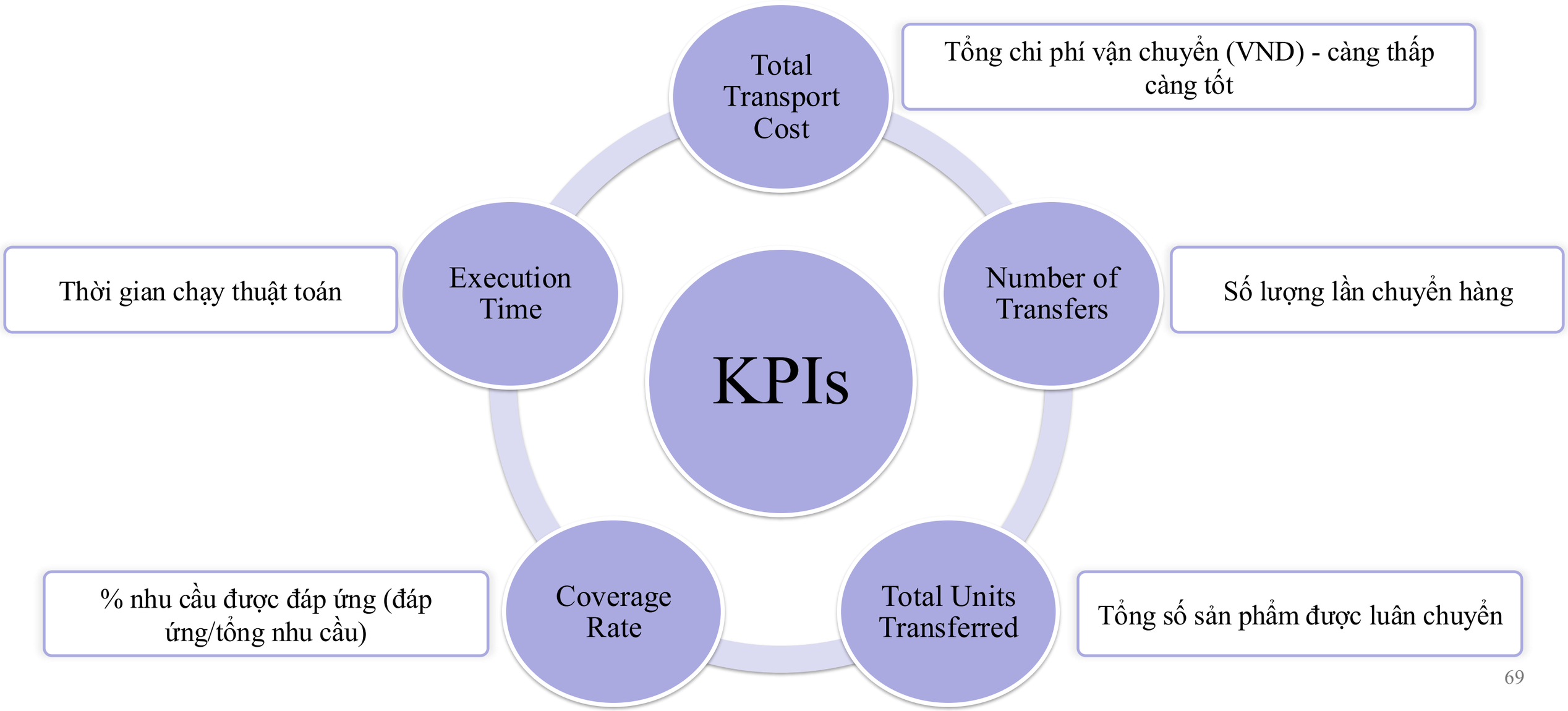
❖ Command Options

```
python src/main.py [OPTIONS]

Options:
  --generate-data          Create sample data for testing
  --rule-based             Use the simple optimization method
  --ga                     Use the advanced optimization method
  --all                    Run both methods and compare results
  --data-dir PATH          Where to find input data (default: data)
  --results-dir PATH       Where to save results (default: results)
  --vis-dir PATH           Where to save charts (default: visualizations)
  --seed INTEGER           Random seed for consistent results (default: 2025)
  --min-days INTEGER       Minimum days of inventory (default: 7)
  --max-days INTEGER       Maximum days of inventory (default: 21)
  --ga-population INTEGER How many solutions to try (default: 100)
  --ga-generations INTEGER How many rounds of improvement (default: 50)
```

Các chỉ số đánh giá (KPIs)

Đo lường hiệu quả vận chuyển thông qua các chỉ số quan trọng



So sánh Rule-Based vs Genetic Algorithm

Rule-Based	Genetic Algorithm
Chi phí: Cao hơn xx-xx%	**Chi phí:** Tối ưu hơn
Thời gian: <xx giây	**Thời gian:** xx-xx phút
Tính ổn định: 100% giống nhau	**Tính ổn định:** Khác nhau mỗi lần chạy
Giải thích: Rất dễ hiểu	**Giải thích:** Khó giải thích
Scalability: Tốt với n lớn	**Scalability:** Chậm với n lớn

Phân tích kết quả - Inventory Analysis

File: results/inventory_analysis.csv

Trạng thái mỗi (store, product): Excess, Balanced, Needed

Days of Supply trước và sau optimization

Số lượng đã chuyển đi/nhận về

Cải thiện: Giảm được bao nhiêu trường hợp thừa/thiếu

Trực quan hóa - Network Graph

Visualize các luồng chuyển hàng:

❖ **Nodes**

Các cửa hàng (Eg. size = tổng lượng chuyển)

❖ **Edges**

Các transfer (Eg. thickness = số lượng, color = chi phí)

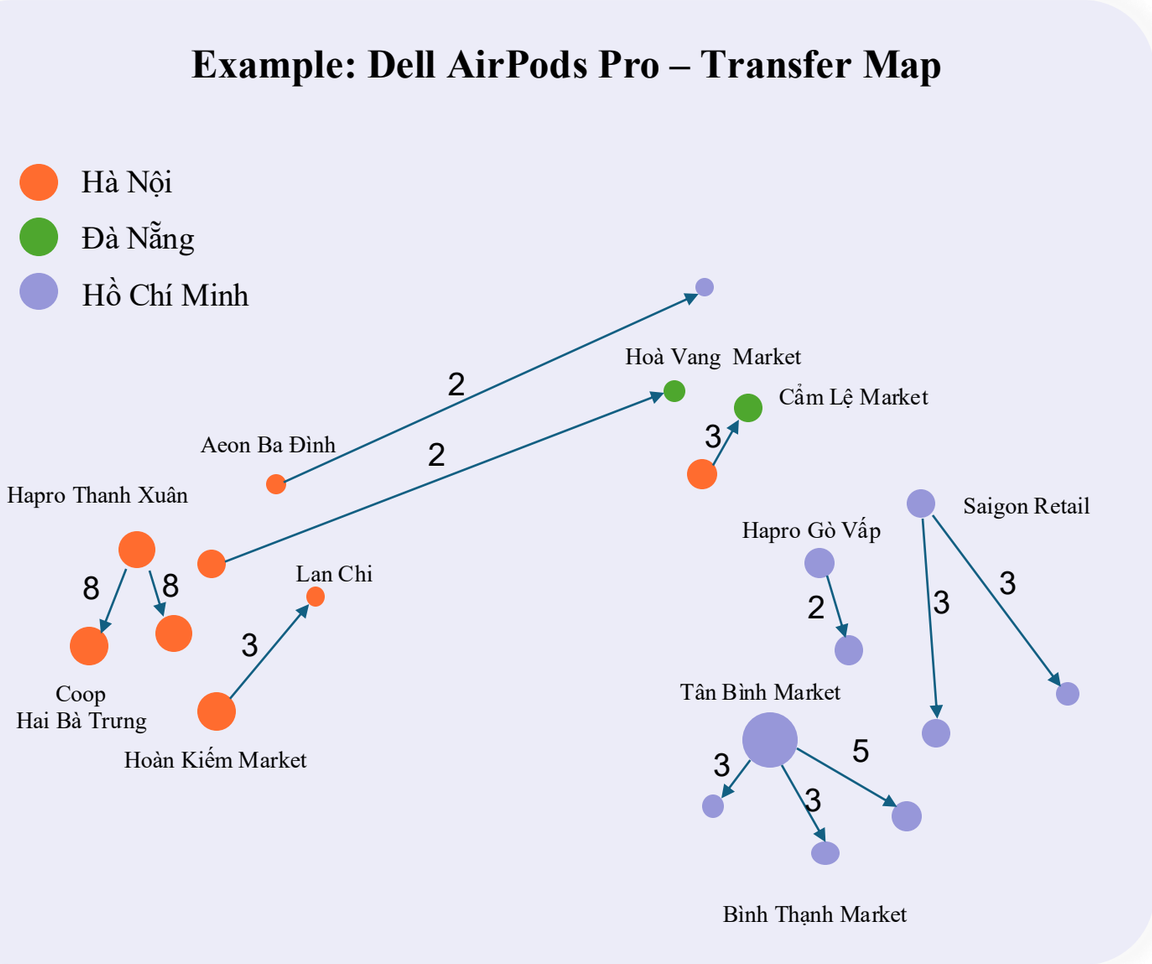
❖ **Layout**

Theo tọa độ địa lý hoặc force-directed

❖ **Tools**

NetworkX, Gephi, Plotly

Output: visualizations/store_transfer_network.png



Dashboard và Báo cáo

File: results/result_summary.txt

Executive Summary
Tổng quan nhanh

Comparison
So sánh 2 thuật toán

Recommendations
Phương án đề xuất

Impact Analysis
Dự báo tác động khi triển khai

Có thể mở rộng: Xây dựng web dashboard với Streamlit/Dash

Đề xuất phương án cải tiến cho dự án

Dự án có thể được cải thiện liên quan đến các khía cạnh như: thuật toán, hệ thống, dữ liệu và mô hình

1. Cải thiện thuật toán

- Nâng cấp GA (hội tụ chậm, local optimum) sang NSGA-II cho tối ưu đa mục tiêu (Chi Phí + Số tuần hàng tồn kho).
- Thử nghiệm các thuật toán khác (Simulated Annealing, PSO, ACO).
- Kết hợp Rule-based với Machine Learning (Hybrid approach).

2. Cải thiện hệ thống

- Xử lý dữ liệu theo thời gian thực (real-time processing).
- Thiết kế microservices để dễ mở rộng (scalability).
- Tích hợp với hệ thống ERP, WMS, POS.
- Xây dựng dashboard trực quan.

3. Cải thiện dữ liệu và mô hình

- Bổ sung yếu tố ngoài (thời tiết, sự kiện, xu hướng thị trường).
- Dự báo nhu cầu chính xác hơn bằng ML (LightGBM, LSTM, Prophet).
- Ràng buộc động theo thời gian thực (dynamic constraints).

Dự đoán nhu cầu bằng AI (Machine Learning)

Lợi ích lớn so với cách dự đoán cũ: Phân tích kỹ hơn, dự đoán đúng hơn vì AI giúp phân tích nhiều thông tin và tìm ra các quy luật ẩn. Nhờ đó có thể dự đoán nhu cầu chính xác hơn, từ đó chủ động lên kế hoạch thay vì chỉ phản ứng khi sự việc xảy ra.

Lợi ích mang lại:

- Giảm nguy cơ hết hàng
- Tiết kiệm chi phí kho bãi
- Ra quyết định nhanh chóng, hiệu quả cao.

Các cách dự đoán:

- **ARIMA:** Xem xét xu hướng và mùa trong năm.
- **LSTM:** Giúp tìm ra mối liên hệ phức tạp, dài hạn.
- **Prophet:** Dùng cho mùa trong năm và các ngày lễ.
- **Boosting:** Kết hợp nhiều mô hình nhỏ để dự đoán tốt hơn (XGBoost).

Các cách dự đoán:

- **Mùa & Xu hướng**: Ví dụ: mùa bán hàng, tốc độ tăng trưởng.
- **Yếu tố bên ngoài:** Thời tiết, tình hình kinh tế chung.
- **Khuyến mãi & Ngày lễ:** Các sự kiện lớn có thể ảnh hưởng.

Ứng dụng nâng cao trong Tối ưu hóa phân bổ hàng hóa

Tận dụng công nghệ AI tiên tiến để tối ưu hóa chuỗi cung ứng và nâng cao hiệu quả phân bổ hàng hóa

AI Agent (LLM base)

Phân tích & Khuyến nghị: Tự động phân tích và đưa ra khuyến nghị phân bổ tối ưu.

Hỗ trợ ra quyết định: Chatbot tương tác, cung cấp gợi ý tức thời cho người quản lý.

Tạo báo cáo tự động: Tự động hóa báo cáo hiệu quả phân bổ, giải thích kết quả tối ưu hóa.

Deep Learning (Học sâu) cho Dự báo nhu cầu

LSTM/GRU: Dự báo nhu cầu phức tạp theo thời gian, nhận diện xu hướng dài hạn và mùa vụ.

CNN: Phân tích mô hình ẩn trong dữ liệu bán hàng, phát hiện nhóm sản phẩm tương đồng.

Transformer models: Dự báo chuỗi thời gian chính xác, hiệu quả với dữ liệu lớn và phức tạp.

Multi-objective Optimization (Tối ưu hóa đa mục tiêu)

Cân bằng nhiều mục tiêu: Tối ưu đồng thời chi phí tồn kho, thời gian giao hàng và mức độ phục vụ.

Pareto optimization: Giúp tìm ra các "sự đánh đổi" tối ưu, hỗ trợ lựa chọn giải pháp phù hợp.

Thuật toán nâng cao: Sử dụng NSGA-II, MOEA/D để giải quyết bài toán phân bổ phức tạp.

Ứng dụng nâng cao trong Tối ưu hóa phân bổ hàng hóa

Tận dụng công nghệ AI tiên tiến để tối ưu hóa chuỗi cung ứng và nâng cao hiệu quả phân bổ hàng hóa

1. Nhược điểm của GA hiện tại:

- Hội tụ chậm, dễ rơi vào local optimum.
- Chỉ tối ưu một mục tiêu duy nhất (chi phí).
- Khó cân bằng giữa các yếu tố khác nhau.

2. Ưu điểm của NSGA-II:

- Tối ưu đa mục tiêu đồng thời.
- Non-dominated sorting để tìm Pareto front.
- Crowding distance để duy trì đa dạng nghiệm.

3. Các mục tiêu tối ưu:

- Minimize: Tổng chi phí vận chuyển.
- Minimize: Tổng lượng hàng vận chuyển.
- Maximize: Service level (đáp ứng nhu cầu).

4. Kết quả mong đợi:

- Nhiều phương án tối ưu để lựa chọn.
- Cân bằng tốt hơn giữa chi phí và hiệu quả.
- Phù hợp với yêu cầu thực tế của doanh nghiệp.

Tổng kết

Tổng kết những kiến thức đã được học và thảo luận trong dự án

1

Những gì đã học được

Hiểu bài toán tối ưu phân bổ hàng hóa.
Nắm vững khái niệm Days of Supply, Excess/Needed Inventory.
Xây dựng mô hình toán học và so sánh Rule-Based với GA.

2

Kỹ năng thực hành

Triển khai thuật toán bằng Python.
Phân tích, đánh giá và trực quan hóa kết quả.

3

Ứng dụng thực tế

Áp dụng vào quản lý chuỗi cửa hàng.
Tiết kiệm chi phí, tối ưu tồn kho.
Hướng phát triển với AI/ML.

4

Kết luận

Tối ưu hóa là công cụ mạnh mẽ trong quản lý doanh nghiệp.
Kết hợp lý thuyết và thực hành giải quyết bài toán thực tế.